

CHAPTER

Functional Programming

3.1	Programs as Functions	47
3.2	Scheme: A Dialect of Lisp	50
3.3	ML: Functional Programming with Static Typing	65
3.4	Delayed Evaluation	77
3.5	Haskell—A Fully Curried Lazy Language with Overloading	81
3.6	The Mathematics of Functional Programming: Lambda Calculus	90

CHAPTER 3

Chapter 1 mentioned several different styles of programming, such as functional programming, logic programming, and object-oriented programming. Different languages have evolved to support each style of programming. Each type of language in turn rests on a distinct model of computation, which is different from the underlying von Neumann model reflected by most programming languages and most hardware platforms. In the next three chapters, we explore each of these alternative styles of programming, the types of languages that support them, and their underlying models of computation. In so doing, we will come to a better understanding of power and limits of languages that reflect the von Neumann model as well.

We begin with functional programming. Unlike other styles of programming, functional programming provides a uniform view of programs as functions, the treatment of functions as data, and the prevention of side effects. Generally speaking, a functional programming language has simpler semantics and a simpler model of computation than an imperative language. These advantages make functional languages popular for rapid prototyping, artificial intelligence, mathematical proof systems, and logic applications.

Until recently, the major drawback to functional languages was inefficient execution. Because of their semantics, such languages at first were interpreted rather than compiled, with a resulting substantial loss in execution speed. In the last 20 years, however, advances in compilation techniques for functional languages, plus advances in interpreter technology in cases where compilation is unsuitable or unavailable, have made functional languages very attractive for general programming. Because functional programs lend themselves very well to parallel execution, functional languages have ironically acquired an efficiency advantage over imperative languages in the present era of multicore hardware architectures (see Chapter 13). Additionally, modern functional languages such as those discussed in this chapter include mature application libraries, such as windowing systems, graphics packages, and networking capabilities. If speed is a special concern for part of a program, you can link a functional program to compiled code from other languages such as C. Thus, there is no reason today not to choose a functional language for implementing complex systems—even Web applications. A functional language is a particularly good choice in situations where development time is short and you need to use clear, concise code with predictable behavior.

Still, although functional languages have been around since the 1950s, they have never become mainstream languages. In fact, with the rise of object orientation, programmers have been even less inclined to use functional languages. Why is that? One possible reason is that programmers typically learn to program using an imperative or object-oriented language, so that functional programming seems inherently foreign and intimidating. Another is that object-oriented languages provide a strong organizing principle for structuring code and controlling complexity in ways that mirror the everyday experience of

real objects. Functional programming, too, has strong mechanisms for controlling complexity and structuring code, but they are more abstract and mathematical and, thus, not as easily mastered by the majority of programmers.

Even if you never expect to write “real” applications in a functional language, you should take the time to become familiar with functional programming. Functional methods, such as recursion, functional abstraction, and higher-order functions, have influenced and become part of many programming languages. They should be part of your arsenal of programming techniques.

In this chapter, we review the concept of a function, focusing on how functions can be used to structure programs. We give a brief introduction to the basic principles of functional programming. We then survey three modern functional languages—Scheme, ML, and Haskell—and discuss some of their properties. We conclude with a short introduction to lambda calculus, the underlying mathematical model for functional languages.

3.1 Programs as Functions

A program is a description of a specific computation. If we ignore the details of the computation—the “how” of the computation—and focus on the result being computed—the “what” of the computation—then a program becomes a virtual black box that transforms input into output. From this point of view, a program is essentially equivalent to a mathematical function.

DEFINITION: A **function** is a rule that associates to each x from some set X of values a unique y from a set Y of values. In mathematical terminology, if f is the name of the function, we write:

$$y = f(x)$$

or

$$f: X \rightarrow Y$$

The set X is called the **domain** of f , while the set Y is called the **range** of f . The x in $f(x)$, which represents any value from X , is called the **independent variable**, while the y from the set Y , defined by the equation $y = f(x)$, is called the **dependent variable**. Sometimes f is not defined for all x in X , in which case it is called a **partial function** (and a function that is defined for all x in X is called **total**).

Programs, procedures, and functions in a programming language can all be represented by the mathematical concept of a function. In the case of a program, x represents the input and y represents the output. In the case of a procedure or function, x represents the parameters and y represents the returned values. In either case, we can refer to x as “input” and y as “output.” Thus, the functional view of programming makes no distinction between a program, a procedure, and a function. It always makes a distinction, however, between input and output values.

In programming languages, we must also distinguish between **function definition** and **function application**. A function definition describes how a value is to be computed using formal

parameters. A function application is a call to a defined function using actual parameters, or the values that the formal parameters assume for a particular computation. Note that, in mathematics, we don't always clearly distinguish between a parameter and a variable. Instead, the term **independent variable** is often used for parameters. For example, in mathematics, we write:

$$\text{square}(x) = x * x$$

for the definition of the squaring function. Then we frequently apply the function to a variable x representing an actual value:

$$\text{Let } x \text{ be such that } \text{square}(x) + 2 \dots$$

In fact, one major difference between imperative programming and functional programming is in how each style of programming treats the concept of a variable. In mathematics, variables always stand for actual values, while in imperative programming languages, variables refer to memory locations that store values. Assignment allows these memory locations to be reset with new values. In mathematics, there are no concepts of memory location and assignment, so that a statement such as

$$x = x + 1$$

makes no sense. Functional programming takes a mathematical approach to the concept of a variable. In functional programming, variables are bound to values, not to memory locations. Once a variable is bound to a value, that variable's value cannot change. This also eliminates assignment, which can reset the value of a variable, as an available operation.

Despite the fact that most functional programming languages retain some notion of assignment, it is possible to create a **pure functional program**—that is, one that takes a strictly mathematical approach to variables.

The lack of assignment in functional programming makes loops impossible, because a loop requires a control variable whose value is reset as the loop executes. So, how do we write repeated operations in functional form? Recursion. For example, we can define a greatest common divisor calculation in C using both imperative and functional form, as shown in Figure 3.1.

```
void gcd( int u, int v, int* x)
{ int y, t, z;
  z = u ; y = v;
  while (y != 0)
  { t = y;
    y = z % y;
    z = t;
  }
  *x = z;
}
```

(a) Imperative version using a loop

```
int gcd( int u, int v)
{ if (v == 0) return u;
  else return gcd(v, u % v);
}
```

(b) Functional version with recursion

Figure 3.1 C code for a greatest common divisor calculation

The second form (Figure 3.1(b)) is close to the mathematical (that is, recursive) definition of the function as

$$\text{gcd}(u, v) = \begin{cases} u & \text{if } v = 0 \\ \text{gcd}(v, u \bmod v) & \text{otherwise} \end{cases}$$

Another consequence of the lack of assignment is that we can have no notion of the internal state of a function. The value of any function depends only on the values of its arguments (and possibly nonlocal variables, which we examine later). For example, the square root of a given number never changes, because the value of the square root function depends only on the value of its argument. The balance of a bank account changes whenever a deposit is made, however, because this value depends not only on the amount deposited but also on the current balance (the internal state of the account).

The value of any function also cannot depend on the order of evaluation of its arguments. This is one reason for using functional programming for concurrent applications. The property whereby a function's value depends only on the values of its arguments (and nonlocal variables) is called **referential transparency**.¹ For example, the gcd function is referentially transparent because its value depends only on the value of its arguments. On the other hand, a function rand, which returns a pseudo random value, cannot be referentially transparent, because it depends on the state of the machine (and previous calls to itself). Indeed, a referentially transparent function with no parameters must always return the same value and, thus, is no different from a constant. In fact, a functional language is free to consider a function of no parameters to not be a function at all.² Referential transparency and the lack of assignment make the semantics of functional programs particularly straightforward. There is no notion of state, since there is no concept of memory locations with changing values. The runtime environment associates names to values only (not memory locations), and once a name enters the environment, its value can never change. These functional programming semantics are sometimes referred to as **value semantics**, to distinguish them from the more usual storage semantics or pointer semantics. Indeed, the lack of local state in functional programming makes it in a sense the opposite of object-oriented programming, where computation proceeds by changing the local state of objects.

Finally, in functional programming we must be able to manipulate functions in arbitrary ways, without arbitrary restrictions—in other words, functions must be general language objects. In particular, functions must be viewed as values themselves, which can be computed by other functions and which can also be parameters to functions. In other words, in functional programming, functions are **first-class data values**.

As an example, one of the essential operations on functions is **composition**. Composition is itself a function that takes two functions as parameters and produces another function as its returned value. Functions like this—that have parameters that are themselves functions or that produce a result that is a function, or both—are called **higher-order functions**.

Mathematically, the composition operator “o” is defined as follows: If $f: X \rightarrow Y$ and $g: Y \rightarrow Z$, then $g \circ f: X \rightarrow Z$ is given by $(g \circ f)(x) = g(f(x))$.

¹A slightly different but equivalent definition, called the substitution rule, is given in Chapter 10.

²Both ML and Haskell (languages discussed later in this chapter) take this approach; Scheme does not: In Scheme, a parameterless function is different from a constant.

We can summarize the qualities of functional programming languages and functional programs as follows:

1. All procedures are functions and clearly distinguish incoming values (parameters) from outgoing values (results).
2. In pure functional programming, there are no assignments. Once a variable is bound to a value, it behaves like a constant.
3. In pure functional programming, there are no loops. Instead, loops are replaced by recursive calls.
4. The value of a function depends only on the value of its parameters and not on the order of evaluation or the execution path that led to the call.
5. Functions are first-class data values.

3.2 Scheme: A Dialect of Lisp

In the late 1950s and early 1960s, a team at MIT led by John McCarthy developed the first language that contained many of the features of modern functional languages. Based on ideas from mathematics, in particular the lambda calculus of Alonzo Church, it was called Lisp (for LISt Processing) because its basic data structure is a list. Lisp, which first existed as an interpreter on an IBM 704, incorporated a number of features that, strictly speaking, are not aspects of functional programming per se, but that were closely associated with functional languages because of the enormous influence of Lisp. These include:

1. The uniform representation of programs and data using a single general data structure—the list.
2. The definition of the language using an interpreter written in the language itself—called a **metacircular interpreter**.
3. The automatic management of all memory by the runtime system.

Unfortunately, no single standard evolved for the Lisp language, and many different Lisp systems have been created over the years. The original version of Lisp and many of its successors lacked a uniform treatment of functions as first-class data values. Also, they used dynamic scoping for nonlocal references. However, two dialects of Lisp that use static scoping and give a more uniform treatment of functions have become standard. The first, Common Lisp, was developed by a committee in the early 1980s. The second, Scheme, was developed by a group at MIT in the mid-1970s. In the following discussion, we use the definition of the Scheme dialect of Lisp given in Abelson et al. [1998], which is the current standard at the time of this writing.

3.2.1 The Elements of Scheme

All programs and data in Scheme are considered expressions. There are two types of expressions: atoms and parenthesized sequences of expressions. Atoms are like the literal constants and identifiers of an imperative language; they include numbers, characters, strings, names, functions, and a few other constructs we will not mention here. A parenthesized expression is simply a sequence of zero or

more expressions separated by spaces and surrounded by parentheses. Thus, the syntax of Scheme is particularly simple:³

```
expression → atom | '(' {expression} ')'
atom → number | string | symbol | character | boolean
```

This syntax is expressed in a notation called **extended Backus-Naur form**, which we will discuss in detail in Chapter 6. For now, note the use of the symbols in the rules shown in Table 3.1.

Table 3.1 Symbols used in an extended Backus-Naur form grammar	
Symbol	Use
→	Means “is defined as”
	Indicates an alternative
{ }	Enclose an item that may be seen zero or more times
' '	Enclose a literal item

Thus, the first rule says that an expression is either an atom or a left parenthesis, followed by zero or more expressions, followed by a right parenthesis. The second rule defines what an atom can be. There are five options shown here, but there are actually several more in the language.

When parenthesized expressions are viewed as data, we call them lists. Some examples of Scheme expressions are shown in Table 3.2.

Table 3.2 Some Scheme expressions	
Expression	What It Represents
42	an integer value
"hello"	a string
#T	the Boolean value “true”
#\a	the character ‘a’
(2.1 2.2 3.1)	a list of real numbers
a	a symbol
hello	another symbol
(+ 2 3)	a list consisting of the symbol + and two integers
(* (+ 2 3) (/ 6 2))	a list consisting of a symbol followed by two lists

The meaning of a Scheme expression is given by an **evaluation rule**. The standard evaluation rule for Scheme expressions is as follows:

1. Atomic literals, such as numbers, characters, and strings, evaluate to themselves.
2. Symbols other than keywords are treated as identifiers or variables that are looked up in the current environment and replaced by the values found there.

³The current Scheme definition (R6RS, Sperber et al. [2009]) gives a more explicit syntactic description that includes some built-in functions and that specifies the precise position of definitions; however, the rules given here still express the general structure of a Scheme program.

An environment in Scheme is essentially a symbol table that associates identifiers with values.

3. A parenthesized expression or list is evaluated in one of two ways:
 - a. If the first item in the expression is a Scheme keyword, such as `if` or `let`, a special rule is applied to evaluate the rest of the expression. Scheme expressions that begin with keywords are called **special forms**, and are used to control the flow of execution. We will have more to say about special forms in a moment.
 - b. Otherwise, the parenthesized expression is a function application. Each expression within the parentheses is evaluated recursively in some unspecified order; the first expression among them must evaluate to a function. This function is then applied to remaining values, which are its arguments.

We can apply these rules to the sample Scheme expressions as follows: `42`, `"hello"`, `#T`, and `#\a` evaluate to themselves; `a` and `hello` are looked up in the environment and their values returned; `(+ 2 3)` is evaluated by looking up the value of `+` in the environment—it returns a function value, namely, the addition function, which is predefined—and then applying the addition function to the values of `2` and `3`, which are `2` and `3` (since constants evaluate to themselves). Thus, the value `5` is returned. Similarly, `(* (+ 2 3) (/ 6 2))` evaluates to `15`. The expression `(2.1 2.2 3.1)`, on the other hand, cannot be evaluated successfully, because its first expression `2.1` is a constant that is not a function. This expression does not represent a Scheme special form or function application and results in an error if evaluation is attempted.

The Scheme evaluation rule implies that all expressions in Scheme must be written in prefix form. It also implies that the value of a function (as an object) is clearly distinguished from a call to the function: The function value is represented by the first expression in an application, while a function call is surrounded by parentheses. Thus, a Scheme interpreter would show behavior similar to the following:

```
> +
#[PROCEDURE: +]

> (+ 2 3)
5
; a call to the + procedure with arguments 2 and 3
```

Note that a Scheme end-of-line comment begins with a semicolon.

A comparison of some expressions in C and Scheme is given in Figure 3.2.

C	Scheme
<code>3 + 4 * 5</code>	<code>(+ 3 (* 4 5))</code>
<code>(a == b) && (a != 0)</code>	<code>(and (= a b) (not (= a 0)))</code>
<code>gcd(10, 35)</code>	<code>(gcd 10 35)</code>
<code>gcd</code>	<code>gcd</code>
<code>getchar()</code>	<code>(read-char)</code>

Figure 3.2 Some expressions in C and Scheme

The Scheme evaluation rule represents **applicative order evaluation**. All subexpressions are evaluated first so that a corresponding expression tree is evaluated from leaves to root. Thus, the Scheme expression `(* (+ 2 3) (+ 4 5))` is evaluated by first evaluating the two additions and then evaluating the resultant expression `(* 5 9)`, as indicated by a bottom-up traversal of the expression tree shown in Figure 3.3.

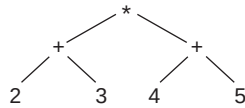


Figure 3.3 Expression tree for Scheme expression

This corresponds to the evaluation of `(2 + 3) * (4 + 5)` in a language like C, where expressions are written in infix form.

Since data and programs in Scheme follow the same syntax, a problem arises when data are represented directly in a program. As noted above, we can represent a list of numbers in Scheme by writing it inside parentheses, with spaces separating the numbers, as in:

```
(2.1 2.2 3.1)
```

However, this list will generate an error if given to the interpreter directly, because Scheme will try to evaluate it as a function call. Thus, the need arises to prevent the evaluation of a list and to consider it to be a list literal (much as simple atomic literals evaluate to themselves). This is accomplished by a special form whose sole purpose is to prevent evaluation. This form uses the Scheme keyword `quote`. The special rule for evaluating a `quote` special form is to simply return the expression following `quote` without evaluating it:

```
> (2.1 2.2 3.1)
Error: the object 2.1 is not a procedure
> (quote (2.1 2.2 3.1))
(2.1 2.2 3.1)
> (quote scheme)
scheme
```

Note that the last expression returns a Scheme **symbol**. Symbols actually form a distinct data type in Scheme. When they are evaluated, symbols are treated as identifiers or names that refer to values. When they are quoted, symbols are taken literally as they are. They can be added to lists or compared for equality to other symbols. The inclusion of symbols as an explicit data type makes Scheme an excellent language for processing any kind of symbolic information, including Scheme programs themselves. We shall have more to say about symbolic information processing later in this section.

The `quote` special form is used so often that there is a special shorthand notation for it—the single quotation mark or apostrophe:

```
> '(2.1 2.2 3.1)
(2.1 2.2 3.1)
```

Because all Scheme constructs are expressions, we need expressions that govern the control of execution. Loops are provided by recursive call, but selection must be given by special forms. The basic forms that do this are the `if` form, which is like an `if-else` construct, and the `cond` form, which is like an `if-elseif` construct. (Note that `cond` stands for **conditional expression**.)

```
(if (= a 0) 0      ; if a = 0 then return 0
    (/ 1 a))      ; else return 1/a

(cond((= a 0) 0)    ; if a=0 then return 0
      ((= a 1) 1)  ; elseif a=1 then return 1
      (else (/ 1 a))) ; else return 1/a
```

The semantics of the expression `(if exp1 exp2 exp3)` dictate that `exp1` is evaluated first; if the value of `exp1` is the Boolean value false (`#f`), then `exp3` is evaluated and its value returned by the `if` expression; otherwise, `exp2` is evaluated and its value is returned. Also, `exp3` may be absent; in that case, if `exp1` evaluates to false, the value of the expression is undefined.

Similarly, the semantics of `(cond exp1 . . . expn)` dictate that each `expi` must be a pair of expressions: `expi = (fst snd)`. Each expression `expi` is considered in order, and the first part of it is evaluated. If `fst` evaluates to `#T` (true), then `snd` is evaluated, and its value is returned by the `cond` expression. If none of the conditions evaluates to `#T`, then the expression in the `else` clause is evaluated and returned as the value of the `cond` expression (i.e., the keyword `else` in a `cond` can be thought of as always evaluating to `#T`). If there is no `else` clause (the `else` clause is optional) and none of the conditions evaluates to `#T`, then the result of the `cond` is undefined.

Notice that neither the `if` nor the `cond` special form obey the standard evaluation rule for Scheme expressions. If they did, all of their arguments would be evaluated each time, regardless of their values. This would then render them useless as control mechanisms. Instead, the arguments to such special forms are **delayed** until the appropriate moment. Scheme function applications use pass by value, while special forms in Scheme and Lisp use delayed evaluation. Delayed evaluation, an important issue in functional programming, is discussed further in Section 3.4.

Another important special form is the `let`, which allows variable names to be bound to values within an expression:

```
> (let ((a 2) (b 3)) (+ a b))
5
```

The first expression in a `let` is a **binding list**, which associates names to values. In this binding list of two bindings, `a` is given the value 2 and `b` the value 3. The names `a` and `b` can then be used for their values in the second expression, which is the body of the `let` and whose value is the returned value of the `let`. Note that `let` provides a local environment and scope for a set of variable names, in much the same manner as temporary variable declarations in block-structured languages such as Java and Python. The values of these variables can be accessed within the `let` form, but not outside it.

To develop a Scheme program, we must also have a way of creating functions. This is done using the `lambda` special form (the term `lambda` comes from Church's lambda calculus). A `lambda` expression has the following form:

```
(lambda param-list body)
```

When Scheme evaluates a `lambda` form, it creates a function with the specified formal parameters and a body of code to be evaluated when that function is applied. For example, a function to compute the area of a circle is:

```
> (lambda (radius) (* 3.14 (* radius radius)))
#<procedure>
```

This function can be applied to an argument by wrapping the function and its argument in another set of parentheses:

```
> ((lambda (radius) (* 3.14 (* radius radius))) 10)
314.0
```

Because it can be useful to refer to functions by names, we can bind a name, such as `circlearea`, to a `lambda` within a `let`, and apply that name to an argument in the body of the `let`:

```
> (let ((circlearea (lambda (radius) (* 3.14 (* radius radius)))))
  (circlearea 10))
314.0
```

Unfortunately, a `let` cannot be used to define recursive functions, since `let` bindings cannot refer to themselves or each other. Thus, the following code will generate an error:

```
(let ((factorial (lambda (n) (if (= n 0) 1 (* n (factorial (- n 1))))))
  (factorial 10))
```

For such situations there is a different special form, called `letrec`, which works just like a `let`, except that it allows arbitrary recursive references within the binding list:

```
(letrec ((factorial (lambda (n) (if (= n 0) 1 (* n (factorial (- n 1))))))
  (factorial 10))
```

The `let` and `letrec` forms introduce variables that are visible within the scope and for the lifetime of the `let` or `letrec`. To make a new, global binding of a variable visible in the top-level environment of Scheme, we can use the `define` special form, as follows:

```
> (define circlearea (lambda (radius) (* 3.14 (* radius radius))))
> (define x 5)
> (circlearea 10)
314.0
> (circlearea x)
78.5
>
```

3.2.2 Dynamic Type Checking

The semantics of Scheme includes dynamic or latent type checking. This means two things: first, only values, not variables, have data types, and second, the types of values are not checked until it is absolutely necessary to do so at runtime. About the only time this happens automatically is right before a so-called primitive function, such as the one named by `+`, is applied to its arguments. If either of these

arguments is not a number, Scheme halts execution with an error message. By contrast, the types of arguments to programmer-defined functions are not automatically checked. For example, the `square` function defined here passes on a potential type error to the `*` function:

```
> (define square (lambda (n) (* n n)))
> (square "hello")
. . *: expects type <number> as 1st argument, given: "hello"
```

The error is indeed caught automatically, but perhaps not at the point where it is most useful. The Scheme programmer can check the type of any value by using any of the built-in type recognition functions, such as `number?` and `procedure?`, in the appropriate conditional expressions. However, forcing the programmer to write code for type checking slows down both the programmer's productivity and the code's execution speed. Later in this chapter, we show how other functional languages, such as ML and Haskell, solve this problem by introducing static type checking.

3.2.3 Tail and Non-Tail Recursion

Because of the runtime overhead required to support procedure calls, loops are always preferable to recursion in imperative languages. However, the functional style of programming encourages recursion. How, then, can we avoid the inefficiency associated with recursion? In languages like Scheme, the answer is to make any recursive steps the last steps in any function. In other words, make your functions **tail recursive**. A Scheme compiler is required to translate such tail recursive functions to code that executes as a loop, with no additional overhead for function calls other than the top-level call. To see how this is so, consider the two definitions of the factorial function in Figure 3.4.

Non-Tail Recursive <code>factorial</code>	Tail Recursive <code>factorial</code>
<pre>> (define factorial (lambda (n) (if (= n 1) 1 (* n (factorial (- n 1)))))) > (factorial 6) 720</pre>	<pre>> (define factorial (lambda (n result) (if (= n 1) result (factorial (- n 1) (* n result))))) > (factorial 6 1) 720</pre>

Figure 3.4 Tail recursive and non-tail recursive functions

Note that *after* each recursive call of the non-tail recursive `factorial`, the value returned by that call must be multiplied by `n`, which was the argument to the previous call. Thus, a runtime stack is required to track the value of this argument for each call as the recursion unwinds. This version of `factorial`, thus, entails a linear growth of memory, a substantial performance hit when compared with a nonrecursive version that uses a loop. However, in the case of the tail recursive version, all the work of computing values is done when the two arguments are evaluated *before* each recursive call. The value of `n` is decremented,

as before. Another argument, `result`, is used to accumulate intermediate products on the way down through the recursive calls. The value of `result` is always 1 on the top-level call, and is multiplied by the current value of `n` before each recursive call. Because no work remains to be done after each recursive call, no runtime stack is really necessary to remember the arguments of previous calls. Thus, the Scheme compiler can translate the tail recursive `factorial` to something like this imaginary code, which uses a loop containing the special forms `begin`, `while`, and `set!`:

```
> (define factorial
  (lambda (n result)
    (begin
      (while (> n 1)
        (set! result (result * n))
        (set! n (- n 1)))
      result)))
```

The `begin` special form forces left-to-right execution of a sequence of expressions and returns the value of the last one. The `set!` special form resets the value of a variable. The `while` special form does not exist in Scheme, but it can be defined with a macro similar to the one discussed in Chapter 2. This of course looks like, and is, imperative code that no bona fide Scheme programmer would ever write. The point is that the programmer can adhere to a recursive, functional style without incurring a performance hit, if the language includes a smart compiler that can generate similar code when optimizing tail recursion.

3.2.4 Data Structures in Scheme

In Scheme, the basic data structure is the list. Although Scheme also supports structured types for vectors (one-dimensional arrays) and strings, a list can represent a sequence, a record, or any other structure. For example, the following is a list representation of a binary search tree:

```
("horse" ("cow" () ("dog" () ()))
 "zebra" ("yak" () () () ))
```

A node in this tree is a list of three items (`name left right`), where `name` is a string, and `left` and `right` are the child trees, which are also lists. Thus, the given list represents the tree of Figure 3.5.

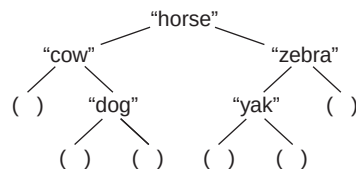


Figure 3.5 A binary search tree containing string data

To use lists effectively we must have an adequate set of functions that operate on lists. Scheme has many predefined list functions, but the basic functions that are common to all Lisp systems are the selector functions `car` and `cdr`, which access the head and the tail of a list, and the constructor function `cons`, which adds a new head to an existing list. Thus, if `L` is the list `(1 2 3)`, then:

```
> (car L)
1
> (cdr L)
(2 3)
> (cons 4 L)
(4 1 2 3)
```

The names `car` and `cdr` are a historical accident. The first machine on which Lisp was implemented was an IBM 704. In that first implementation, addresses or pointers were used to represent lists in such a way that the head of a list was the “Contents of the Address Register,” or `car`, and the tail of a list was the “Contents of the Decrement Register,” or `cdr`. This historical accident persists partially because of another accident: On account of the single letter difference in the names of the operations, repeated applications of both can be combined by combining the letters “a” and “d” between the “c” and the “r.” Thus, `(car (cdr L))` becomes `(cadr L)`, `(cdr (cdr L))` becomes `(cddr L)`, and `(car (cdr (cdr L)))` becomes `(caddr L)`. For clarity, however, we will avoid these abbreviations.

The view of a list as a pair of values represented by the `car` and the `cdr` has also continued to be useful as a representation or visualization of a list. According to this view, a list `L` is a pointer to a “box” of two pointers, one to its `car` and the other to its `cdr`. See Figure 3.6.

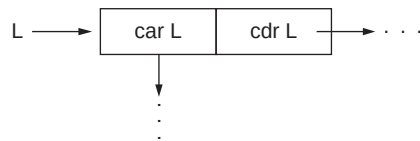


Figure 3.6 Visualizing a list with box and pointer notation

This **box and pointer notation** for a simple list such as `(1 2 3)` is shown in Figure 3.7.

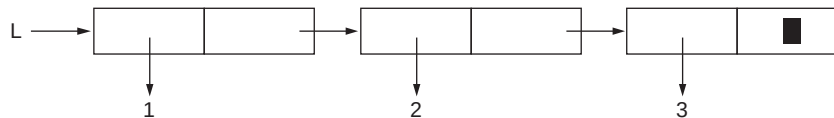


Figure 3.7 Box and pointer notation for the list `(1 2 3)`

The symbol black rectangle in the box at the end stands for the empty list `()`. A more complex example of a list structure is shown in Figure 3.8.

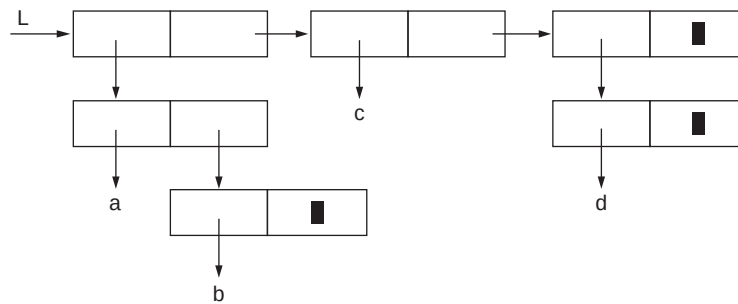


Figure 3.8 Box and pointer notation for the list $L = ((a\ b)\ c\ (d))$

A box and pointer diagram is useful for interpreting the list operations: The `car` operation represents following the first pointer, while the `cdr` operation represents following the second pointer. Thus, for the L of Figure 3.8, $(\text{car } (\text{car } L)) = a$, $(\text{cdr } (\text{car } L)) = (b)$, and $(\text{car } (\text{cdr } (\text{cdr } L))) = (d)$.

All the basic list manipulation operations can be written as functions using the primitives⁴ `car`, `cdr`, `cons`, and `null?`. The last function is a predicate that returns true if its list argument is empty, or false otherwise. For example, an `append` operation that returns the appended list of two lists can be written as follows:

```
(define append (lambda (L M)
  (if (null? L) M
      (cons (car L) (append (cdr L) M))))
```

and a `reverse` operation is as follows:

```
(define reverse (lambda (L)
  (if (null? L) '()
      (append (reverse (cdr L)) (list (car L)))))
```

In the `reverse` function, we use the primitive function `list`, which makes a list of one or more items:

```
> (list 2 3 4)
(2 3 4)
> (list '(a b) '(c d))
((a b) (c d))
```

In fact, in the `reverse` function, we could have used $(\text{cons } (\text{car } L) \text{ '()})$ instead of $(\text{list } (\text{car } L))$, since $(\text{list } a)$ is the same as $(\text{cons } a \text{ '()})$.⁵

⁴A primitive is a built-in procedure that is implemented at a lower level. Most built-in procedures required by the Scheme standard are primitives, so we will not emphasize this distinction.

⁵Actually, `append` and `reverse` are also really built-in primitives, although as we have seen, they could be implemented in Scheme itself.

Finally, we offer the example of the use of `car` and `cdr` to access the elements of a binary search tree defined as a list with structure `(data leftchild rightchild)`:

```
(define (leftchild B) (car (cdr B)))
(define (rightchild B) (car (cdr (cdr B))))
(define (data B) (car B))
```

Now we can write a tree traversal procedure `print-tree` that prints out the data in the tree as follows:

```
(define print-tree (lambda (B)
  (cond ((null? B) '())
        (else (print-tree (leftchild B))
                (display (data B))
                (newline)
                (print-tree (rightchild B))))))
```

3.2.5 Programming Techniques in Scheme

Programming in Scheme, as in any functional language, relies on recursion to perform loops and other repetitive operations. One standard technique for applying repeated operations to a list is to “cdr down and cons up”—meaning that we apply the operation recursively to the tail of a list and then collect the result with the `cons` operator by constructing a new list with the current result. One example is the `append` procedure code of the previous section. Another example is a procedure to square all the members in a list of numbers:

```
(define square-list (lambda (L)
  (if (null? L) '()
      (cons (* (car L) (car L)) (square-list (cdr L))))))
```

An example of a loop that is not applied to a list is a procedure that simply prints out the squares of integers from a lower bound to an upper bound:

```
(define print-squares (lambda (low high)
  (cond ((> low high) '())
        (else (display (* low low))
                (newline)
                (print-squares (+ 1 low) high)))))
```

A call to `(print-squares 1 100)` will generate the desired result.

In the `print-squares` example, we had to use the extra parameter `low` to control the recursion, just as a loop index controls repetition. Earlier we noted that, in such cases, tail-recursive procedures are preferred because of the relative ease with which translators can optimize them into actual loops. In fact, the `print-squares` function is tail recursive.

In the last section, we also demonstrated the technique of using an accumulating parameter to turn a non-tail-recursive procedure into a tail-recursive one. As an example in Scheme, we apply this technique

to the `square-list` function by defining a helping procedure `square-list1` with an extra parameter to accumulate the intermediate result:

```
(define square-list1 (lambda (L list-so-far)
  (if (null? L) list-so-far
      (square-list1 (cdr L)
                    (append list-so-far (list (* (car L) (car L)))))))
```

Now we define `square-list` as a call to `square-list1` with `()` as its first accumulated value:

```
(define square-list (lambda (L) (square-list1 L '())))
```

Similarly, for reverse one defines `reverse1` and `reverse` as follows:

```
(define reverse1 (lambda (L list-so-far)
  (if (null? L) list-so-far
      (reverse1 (cdr L) (cons (car L) list-so-far))))

(define reverse (lambda (L) (reverse1 L '())))
```

3.2.6 Higher-Order Functions

Since functions are first-class values in Scheme, we can write functions that take other functions as parameters and functions that return functions as values. Such functions are called **higher-order functions**.

To give a simple example of a function with a function parameter, we can write a function `map`⁶ that applies another function to all the elements in a list and then pass it the `square` function to get the `square-list` example:

```
(define map (lambda (f L)
  (if (null? L) '()
      (cons (f (car L)) (map f (cdr L)))))

(define square (lambda (x) (* x x)))

(define square-list (lambda (L) (map square L)))
```

Here is an example of a function that has a function parameter and also returns a function value:

```
(define make-double (lambda (f)
  (lambda (x) (f x x))))
```

This function assumes that `f` is a function with two parameters and creates the function that repeats the parameter `x` in a call to `f`. The function value `doublefn` returned by `make-double` is created by a

⁶The `map` function is in fact a standard Scheme function, with behavior somewhat more general than what we describe here.

local `define` and then written at the end to make it the returned value of `make-double`. Note that `x` is not a parameter to `make-double` itself but to the function created by `make-double`.

We can use `make-double` to get both the `square` function and the `double` function (a function that doubles its numerical value):

```
(define square (make-double *))
(define double (make-double +))
```

The symbols “`*`” and “`+`” are the names for the multiplication and addition functions. Their values—which are function values—are passed to the `make-double` procedure, which in turn returns function values that are assigned to the names `square` and `double`. Note that we are using here the simple form of the `define`, which just assigns computed values rather than defining a function with a given body. Just as

```
(define a 2)
```

assigns the value 2 to the name `a`, the expression

```
(define square (make-double *))
```

assigns the function value returned by `make-double` to the name `square`.

As noted at the end of Section 3.2, this ability of functional languages, including Scheme, to return function values from higher-order functions means that the runtime environment of functional languages is more complicated than the stack-based environment of a standard block-structured imperative language. Returning the memory used by functions to free storage requires the use of automatic memory management techniques, such as **garbage collection**, which are discussed in Section 10.5.

3.2.7 Static (Lexical) Scoping

Early dialects of Lisp were dynamically scoped, whereas more modern dialects, such as Scheme and Common Lisp, are statically scoped. The term **static scope**, or **lexical scope**, refers to the area of a program text in which a declaration of a variable is visible. Thus, in static scoping, the meaning or value of a given variable can be determined just by reading the text of the program. In a dynamically scoped language, by contrast, the meaning of a variable depends on the runtime context.

The concept of scope is important in block-structured languages, where it is possible to nest the declarations of variables. Consider, for example, the following Scheme code segment, which nests a `let` within a `let`:

```
> (let ((a 2) (b 3))
    (let ((a (+ a b)))
      (+ a b)))
8
```

The scope rule states that the scope of a variable extends to the end of the block in which it is declared, including any nested blocks, unless that variable is redeclared within a nested block. Thus, in this example, the value of the first instance of `(+ a b)` is 5, whereas the value of the second instance of `(+ a b)` is 8.

This straightforward application of the scope rule becomes more complicated when we include functions with free variables. A **free variable** is a variable referenced within a function that is not also a formal parameter to that function and that is not bound within a nested function. A **bound variable** is a variable within a function that is also a formal parameter to that function. Here is an example of the definition and use of a function that contains a free variable and a bound variable:

```
> (let ((pi 3.14))
    (let ((circlearea (lambda (radius)
                        (* pi (* radius radius)))))
      (let ((pi 3.1416))
        (circlearea 10))))
314.0
```

The `circlearea` function contains one bound variable, `radius`, and one free variable, `pi`. The bound variable `radius` is also the function's formal parameter, and it picks up its value in the context of its function's call, when it is bound to the value 10. You can also see that the value of the free variable `pi` must have been 3.14 when the function referenced it during this application. Thus, the free variable `pi` picks up and retains its value in the outer `let`, which is the context of the function's definition, rather than in the inner `let`, which is the context of the function's call. By contrast, if Scheme were dynamically scoped, like older Lisps, the value of `pi` used in the function application would be the one given it by the inner `let`, or the context of the function's call. The application would then have produced the value 314.16. In that case, the function's free variable would have the same semantics as its formal parameter. That is, in dynamic scoping, the values of free variables and formal parameters vary with the context of the function's call.

Lexical scoping, because it fixes the meaning of free variables in one place in the code, generally makes a program easier to read and verify than dynamic scoping.

3.2.8 Symbolic Information Processing and Metalinguistic Power

Earlier in this section, we showed that Scheme programs and Scheme data can have the same syntax. For example, viewed as data, the expression `(f x y)` is just a list of three symbols, which can be accessed and manipulated with list operations. When viewed as a program, this expression is the application of a function named by the variable `f` to its argument values named by the variables `x` and `y`. The capacity to build, manipulate, and transform lists of symbols and then to evaluate them as programs gives the Scheme programmer the power to create new languages for special purposes. Abelson and Sussman [1996] call this type of power **metalinguistic** and give several examples of its use. Among them are the creation of a constraint language for expressing relations rather than functions, and the construction of a language that supports logical inference.

The creation of these little languages requires the programmer to specify the syntax of their expressions and their semantics as realized in new interpreters, all written in the base language Scheme. While building a new interpreter might sound like a much more daunting task than simply defining new functions or data types, the job is made surprisingly simple by many of the features of Scheme discussed thus far in this section.

To give the flavor of this idea, let's examine how a Scheme programmer might process a `let` expression if Scheme did not include `let` as a built-in special form. Recall that `let` allows the interpreter to evaluate an expression within a new environment created by binding a set of temporary variables to their values. A close inspection, however, shows that the `let` form is actually just syntactic sugar for the application of a `lambda` form to its arguments. Figure 3.9 shows examples of a `let` expression and a `lambda` application that have the same semantics.

```
(let ((a 3) (b 4))      ((lambda (a b) (* a b)) 3 4)
  (* a b))
```

Figure 3.9 `let` as syntactic sugar for the application of `lambda`

As you can see, the temporary variables of the `let` become the formal parameters of the `lambda`, and their initial values become the arguments of the `lambda`'s application. The body of the `let` becomes the body of the `lambda`.

The process of interpreting a `let` form that is not built-in involves two steps. First, we treat the `let` form as a datum (a list) to be transformed into another datum (another list) that represents the equivalent `lambda` application. Second, we evaluate the new representation. If we assume that there exists a function, `let-to-app`, that performs the required syntactic transformation, then we can apply the Scheme function `eval` to the result to evaluate it, as follows:

```
> (eval (let-to-app '(let ((a 3) (b 4))(* a b))))
12
```

Note that we quote the argument of `let-to-app`, the `let` expression, to treat it as data (a list of elements). We then use `eval` to force evaluation of the result of `let-to-app`, which is another list of elements representing the application of the `lambda`.

The remaining task is to define the function `let-to-app`, which transforms any `let` expression to the corresponding `lambda` application. Our implementation makes use of several list-processing functions and the quote of a symbol. The comments indicate the phrases of code extracted from the `let` form and added to the list representing the application under construction:

```
(define let-to-app (lambda (let-form)
  (append (list                                     ; begin the app
          (list 'lambda                             ; begin the lambda
              (map car (cadr let-form))              ; formal parameters
              (caddr let-form)))                     ; lambda body
          (map cadr let-form))))                     ; argument expressions
```

Alternatively, we could define the syntactic forms of our new language (if they are not already built in) using a macro facility as mentioned in Chapter 2. Then the programmer using the new forms would not have to quote them and explicitly evaluate them with `eval`. Upon encountering the new forms, the Scheme compiler would automatically perform the required syntactic transformations and generate code that is evaluated in the usual manner at runtime.

3.3 ML: Functional Programming with Static Typing

As we have noted previously, ML (for MetaLanguage) is a functional programming language that is quite different from the dialects of Lisp, such as Scheme. First, ML has a more Algol-like syntax, which avoids the use of many parentheses. Second, it is statically typed, so that the type of every expression is determined before execution, and types can be checked for consistency. While traditional Lisp programmers may dislike the constraints of a static type system, ML's static type system offers significant advantages. For one thing, it makes the language more secure, because more errors can be found prior to execution. This is especially important in instructional settings and also helps ensure good software engineering. Also, ML's static type system improves efficiency because: (1) it makes it possible to predetermine size and allocation requirements, and (2) it makes checking types at runtime unnecessary. Additionally, ML's static type system, which you will study in detail in Chapter 8, is extremely flexible. It does not insist, as with C or Ada, that *all* types be declared by the programmer, and it also allows for parametric polymorphism, where types can contain type variables that can range over the set of all types.

ML was first developed in the late 1970s as part of a larger system for proving the correctness of programs, known as the Edinburgh Logic for Computable Functions (LCF) system, developed by a team led by Robin Milner. Milner also helped develop the strong type inference system, based on pattern matching, that is now called Hindley-Milner type checking. This form of type checking is a key feature of both ML and Haskell (discussed later in this chapter). During a revision of the ML language during the 1980s, features of the earlier language were combined with the HOPE language, developed by Rod Burstall, also at Edinburgh. This new language was named Standard ML, or SML. Another minor revision to the language occurred in 1997, and the current standard is referred to as SML97 or just ML97. We use that revision in this text.

In this section, we will first review the basics of ML, including comparisons with Scheme. We will then provide an introduction to ML data structures and programming techniques. (Some ML data structure information will be covered in Chapter 8.)

3.3.1 The Elements of ML

In ML, the basic program is, as in Scheme, a function declaration, as in:

```
> fun fact n = if n = 0 then 1 else n * fact(n - 1);
val fact = fn: int -> int
```

The reserved word `fun` in ML introduces a function declaration. The identifier immediately after `fun` is the name of the function, and the names of the parameters follow, up to the equal sign. After the equal sign is the body of the function.

The ML system responds to a declaration by returning the data type of the value defined. In this example, `fact` is a function, so ML responds that the value of `fact` is `fn`, with type `int -> int`, which means that `fact` is a function from integers (its domain) to integers (its range). Indeed, we could have given type declarations for the type of `fact` and the type of its parameter as follows:

```
> fun fact (n: int): int = if n = 0 then 1
  else n * fact (n - 1);
val fact = fn: int -> int
```

Once a function has been declared it can be called as follows:

```
> fact 5;
val it = 120 : int
```

ML responds with the returned value and its type (`it` is the name of the current expression under evaluation). ML has essentially the same evaluation rule as Scheme: `fact` must evaluate to a function, then 5 is evaluated, and its type must agree with the parameter type of the function. The function is then called and its returned value printed together with its type. ML, however, does not need a quote function as in Scheme to prevent evaluation, because data are distinct from programs. In ML, parentheses are almost completely unnecessary, because the system can determine the meaning of items based solely on their position.

One can also define values in ML by using the `val` keyword:

```
> val Pi = 3.14159;
val Pi = 3.14159 : real
```

In ML, arithmetic operators are written as infix operators as they are in C or Ada. This differs from the uniform prefix notation of Lisp. As a result, operator precedence and associativity are an issue in ML. Of course, ML adheres to the standard mathematical conventions for the usual arithmetic operators. For example, $2 + 3 * 4$ means $2 + (3 * 4)$.⁷ In ML, it is also possible to turn infix operators into prefix operators using the `op` keyword, so that $2 + 3 * 4$ can be written as:

```
> op + (2 , op * ( 3,4));
val it = 14 : int
```

Note here that the binary arithmetic operators take *pairs* of integers as their arguments. These pairs are elements of the Cartesian product type, or **tuple type** `int * int`:

```
> (2,3);
val it = (2,3) : int * int
> op +;
val it = fn : int * int -> int
```

Lists are an important feature of ML, as they are in almost every functional language. However, they are not as universal as they are in Lisp, because in ML programs are not themselves lists. A list in ML is indicated by square brackets, with elements separated by commas. For example, a list of the integers 1, 2, and 3 is represented as `[1, 2, 3]`:

```
> [1,2,3];
val it = [1,2,3] : int list
```

⁷On the other hand, the unary minus or negation operator, which is usually written in mathematics with the same `-` symbol as subtraction, is written using a tilde in ML: `~5` is negative 5.

ML determines the type of this list to be `int list`. ML's strong typing dictates that lists may only contain elements that all have the same type. Here is an example structure that violates this restriction by mixing types of elements in a list:

```
> [1,2,3.1];
Error: operator and operand don't agree [literal]
  operator domain: int * int list
  operand:         int * real list
in expression:
  2 :: 3.1 :: nil
```

If we want to mix data of different types, we must use a **tuple**:

```
> (1,2,3.1);
val it = (1,2,3.1) : int * int * real
```

The error message generated in the earlier example also gives us a hint as to how ML constructs lists, which is similar to the way Scheme constructs lists. That is, the operator `::` corresponds to `cons` in Scheme, and constructs a list out of an element (which becomes the head of the list) and a previously constructed list (which becomes the tail of the list). For example:

```
> op :: ;
val it = fn : 'a * 'a list -> 'a list
> 2 :: [3,4];
val it = [2,3,4] : int list
```

Note that the type of the `::` operator contains a **type variable** `'a`, because `::` can be applied to a list containing values of any type. Thus, `::` is a function from any type `'a` and any list of the same type `'a` to a list of the same type `'a`.

In ML, every list is constructed by a series of applications of the `::` operator:

```
> 1 :: 2 :: 3 :: [];
val it = [1,2,3] : int list
```

Here `[]` is the empty list, which can also be written as the keyword `nil`.

ML has operations corresponding to Scheme's `car` and `cdr` operators as well, except that they are called `hd` and `tl` (for **head** and **tail**). For example:

```
> hd [1,2,3];
val it = 1 : int
> tl [1,2,3];
val it = [2,3] : int list
```

In fact, these functions are used much less in ML than their equivalents in Scheme, because ML's pattern-matching ability makes them unnecessary. In ML, if we want to identify the head and tail of a list L , we simply need to write $h :: t$ for the list. ML will then extract h as the head and t as the tail by matching the pattern against the list L . If L is empty, the pattern will not match, and the following warning will be generated:

```
> val h::t = [1,2,3];
Warning: binding not exhaustive
      h :: t = ...
val h = 1 : int
val t = [2,3] : int list
```

Pattern matching is more typically used in function definitions and in case expressions. For example, a list append function in ML can be defined as follows:⁸

```
> fun append ([], L) = L
    | append (h :: t, L) = h :: append (t, L);
val append = fn : 'a list * 'a list -> 'a list
```

This is equivalent to the following definition using a case expression:

```
> fun append (x,y) =
    case x of
      [] => y |
      (h::t) => h :: append (t,y);
```

Pattern matching can be used in virtually all function definitions where a case analysis is called for, and it can eliminate most uses of `if` expressions. For example, the recursive factorial function defined previously can also be written using pattern matching:

```
fun fact 0 = 1 | fact n = n * fact (n - 1);
```

Here the second pattern—the variable n —matches anything, but its position as the second pattern implies that it must not be zero; if it were, the first pattern (the integer literal 0) would have matched.

Patterns in ML can also contain wildcards, written as the underscore character. Wildcards are patterns that match anything, but whose contents we don't care about. For example, we could define our own version of the `hd` function as follows:

```
fun hd (h::_) = h;
```

In fact, this definition will generate a warning message (“match nonexhaustive”), since we have not included a definition of what `hd` is to do in the case of the empty list. A better definition is the following, which is essentially exactly what the predefined `hd` does:

```
fun hd (h::_) = h | hd [] = raise Empty;
```

⁸In fact, ML has a built-in infix append operator `@`: $[1,2,3] @ [4,5,6] = [1,2,3,4,5,6]$.

ML's strong typing, while secure and flexible, does impose important constraints on what is an acceptable program. A typical example is a square function, which we define here for real (floating point) values:

```
> fun square x: real = x * x;
val square = fn : real -> real
```

An error will now be generated if we try to call this function with an integer, since ML (like ADA) will not automatically convert an integer into a real:

```
> square 2;
Error: operator and operand don't agree [literal]
operator domain: real
operand:         int
in expression:
square 2
```

Instead, we need to manually convert the integer using a conversion function. In the following example, the function has the same name as the data type we are converting to:

```
> square (real 2);
val it = 4.0 : real
```

We might think that we could define a similar function with the same name for integers:

```
> fun square x: int = x * x;
val square = fn : int -> int
```

but ML does not allow overloading. Thus, this definition will just replace the previous one. We might also try defining a generic version of square by not specifying the type of the parameter:

```
> fun square x = x * x;
val square = fn : int -> int
```

In this case, however, ML simply assumes that we meant `x` to be an integer.⁹ The problem is that, while the built-in arithmetic functions are overloaded for arithmetic types like `int` and `real`, we cannot make use of that fact to define any overloaded user-defined functions. For the most part, this is not a problem in the language Haskell (as explained later in this chapter).

A related issue is that certain values in ML do not contain enough type information for ML to properly type check an expression before it is evaluated. The main example of this is the empty list `[]`, which has type `'a list`. If ML is to evaluate a polymorphic function on the empty list, ML will, in certain cases, indicate that not enough information is known about the type of `[]`:

```
> rev [];
type vars not generalized because of value
restriction are instantiated to dummy types (X1,X2,...)
val it = [] : ?X1 list
```

⁹Older versions of ML made this function definition into an error.

Here `rev` is the built-in reverse function that reverses a list, and has type `'a list -> 'a list`. However, when `rev` is applied to the empty list, ML wants to instantiate the type variable `'a` to a specific type, and it cannot because not enough information is available. Nevertheless, ML compensates by supplying a “dummy” type, and gets the right answer, despite the warning message. If we wish, supplying a specific type avoids the problem:

```
- rev []:int list;
val it = [] : int list
```

A further unusual type-checking phenomenon is that ML makes a strong distinction between types that can be compared for equality and types that cannot. In particular, real numbers cannot be compared for equality:

```
> 2.1 = 2.1;
Error: operator and operand don't agree [equality type required]
operator domain: 'Z * 'Z
operand:         real * real
in expression:
  2.1 = 2.1
```

When a polymorphic function definition involves an equality comparison, the type variable(s) can only range over the **equality types**, which are written with two quotes rather than a single quote:

```
> fun constList [] = true
    | constList [a] = true
    | constList (a::b::L) = a = b andalso constList (b::L);
val constList = fn : ''a list -> bool
```

Indeed, the equality function itself can only be applied to equality types:

```
> op =;
val it = fn : ''a * ''a -> bool
```

This somewhat awkward state of affairs has an elegant solution in Haskell, as you will see in the next section.

We conclude this section with a brief description of simple input and output in ML, and write an ML version of the Euclid program written in Scheme earlier in the chapter.

ML’s version of the library package is the **structure** (see Chapter 11). It includes several standard predefined resources that are useful in performing input and output. The principle I/O functions are in the `TextIO` structure, and the simplest of these are the `inputLine` and `output` functions¹⁰:

¹⁰We are ignoring the simpler `print` function in the interest of uniformity.

```

> open TextIO; (* dereference the TextIO structure *)
... (* ML interpreter lists contents of TextIO *)
> output(stdout, inputLine(stdin));
      (* reads and writes a line of text *)
Hello, world!
Hello, world!
val it = () : unit
>

```

Note that the returned value of an output operation is the value `()`, which is of type `unit`. This is similar to the `void` type of C, but in ML every function must return *some* value, so the `unit` type has exactly one value `()` that represents “no actual value.”¹¹

Now, these two I/O functions only read and write strings. To convert between strings and numbers, we must use the `toString` and `fromString` functions from the utility structures for the appropriate types (for example, the `Int` structure for ints):

```

> Int.toString;
val it = fn : int -> string
> Int.fromString;
val it = fn : string -> int option

```

This presents yet another small hurdle. The `fromString` function may fail to extract any number, in which case it returns a value of the `option` type. This is defined as follows:

```
datatype 'a option = NONE | SOME of 'a;
```

Finally, when performing a series of input and output operations, it is convenient to collect these operations in an **expression sequence** (like the `begin` construct of Scheme), which is a semicolon-separated sequence of expressions surrounded by parentheses, and whose value is the value of the last expression listed:

```

> fun printstuff () =
    ( output(stdout, "Hello\n");
      output(stdout, "World!\n")
    );
val printstuff = fn : unit -> unit
> printstuff ();
Hello
World!
val it = () : unit

```

¹¹`Unit` can also be used to imitate parameterless functions in ML, since ML requires every function to have at least one parameter.

Now we are ready for the `euclid` program in ML¹²:

```
> fun gcd (u,v) = if v = 0 then u
                    else gcd (v, u mod v) ;
val gcd = fn : int * int -> int
> fun euclid () =
  ( output(stdOut, "Enter two integers:\n");
    flushOut(stdOut);
    let val u = Int.fromString(inputLine(stdIn));
        val v = Int.fromString(inputLine(stdIn))
    in
      case (u,v) of
        (SOME x, SOME y) =>
          ( output(stdOut,"The gcd of ");
            output(stdOut,Int.toString(x));
            output(stdOut," and ");
            output(stdOut,Int.toString(y));
            output(stdOut," is ");
            output(stdOut,Int.toString(gcd(x,y)));
            output(stdOut,"\n")
          ) |
        _ => output(stdOut, "Bad input.\n")
      end
    )
val euclid = fn : unit -> unit
> euclid ();
Enter two integers:
15
10
The gcd of 15 and 10 is 5
val it = () : unit
```

3.3.2 Data Structures in ML

Unlike Scheme, ML has a rich set of data types, from enumerated types to records to lists. You have already seen lists and tuples (Cartesian products) above. In Chapter 8, you will see additional examples of type structures in ML. We will preview some of those and examine some additional structures briefly here. While Scheme relies primarily on lists to simulate other data structures, ML has many options for user-defined data types, similar to C++ or Ada.

¹²There is one further quirk to be mentioned: We must use the `flushOut` function after the prompt string to make sure its printing is not delayed until after the input.

First, it is possible to give synonyms to existing data types using the keyword `type`:

```
> type Salary = real;
type Salary = real
> 35000.00: Salary;
val it = 35000.0 : Salary
> type Coords = real * real;
type Coords = real * real
> (2.1,3.2):Coords;
val it = (2.1,3.2) : Coords
```

ML also has new user-defined data types, introduced by the `datatype` keyword. For example, an enumerated type can be defined as follows:

```
> datatype Direction = North | East | South | West;
```

The vertical bar is used for alternative values in the declaration and the names, such as `North` and `East`, are called **value constructors** or **data constructors**. Data constructors can be used as patterns in declarations or case expressions, as in the following function definition:

```
> fun heading North = 0.0 |
    heading East = 90.0 |
    heading South = 180.0 |
    heading West = 270.0 ;
val heading = fn : Direction -> real
```

Recursive types such as binary search trees can also be declared using a `datatype` declaration:

```
> datatype 'a BST = Nil | Node of 'a * 'a BST * 'a BST;
```

Note that this `datatype` is parameterized by the type of data stored in the tree. We can now construct the tree of Figure 3.5 as follows:

```
> val tree = Node("horse",
    Node( "cow", Nil, Node("dog",Nil,Nil) ),
    Node("zebra",Node("yak",Nil,Nil), Nil));
val tree = Node ("horse",Node ("cow",Nil,Node #),Node ("zebra",Node #,Nil))
: string BST
```

We can define `data`, `leftchild`, and `rightchild` functions on BSTs by pattern matching. For example,

```
> fun leftchild (Node(data,left,right)) = left
    | leftchild Nil = raise Empty;
val leftchild = fn : 'a BST -> 'a BST
```

A traversal function for binary search trees that prints out the string nodes is as follows:

```
> fun print_tree Nil = () |
    print_tree (Node (data, left, right)) =
        ( print_tree left;
          output(stdOut, data);
          output(stdOut, "\n");
          print_tree right);
val print_tree = fn : vector BST -> unit
> print_tree tree;
cow
dog
horse
yak
zebra
val it = () : unit
```

3.3.3 Higher-Order Functions and Currying in ML

Higher-order functions and expressions whose values are functions are as useful in ML as in Scheme.

ML's keyword for a function expression, `fn`, is used with a special arrow (an equals followed by a greater than) to indicate the body of the function. For example:

```
> fn x => x * x;
val it = fn : int -> int
> (fn x => x * x) 4;
val it = 16 : int
```

A `fn` expression can be used to build anonymous functions and function return values. Indeed a `fun` definition is, as in Scheme, just syntactic sugar for the use of a `fn` expression:

```
fun square x = x * x;
```

is equivalent to:

```
val square = fn x => x * x;
```

One difference between ML and Scheme is that, if we use an ML `fn` expression to declare a recursive function, we must add the `rec` keyword in ML (similar to a Scheme `letrec`):

```
val rec fact = fn n => if n = 0 then 1 else n * fact (n - 1);
```

Function expressions can be used to create function return values as well as function arguments, as, for example, a `make_double` function that repeats an argument to a two-parameter function:

```
> fun make_double f = fn x => f (x,x);
val make_double = fn : ('a * 'a -> 'b) -> 'a -> 'b
> val square = make_double (op * );
val square = fn : int -> int
> val double = make_double (op +);
val double = fn : int -> int
> square 3;
val it = 9 : int
> double 3;
val it = 6 : int
```

ML has a number of built-in higher-order functions, including function composition, given by the letter `o` (lowercase letter “O”). For example:

```
> val double_square = double o square;
val double_square = fn : int -> int
> double_square 3;
val it = 18 : int
```

Higher-order functions and expressions whose values are functions are made even more useful in ML through a process known as currying. A function of multiple parameters that can be viewed as a higher-order function of a single parameter that returns a function of the remaining parameters is said to be **curried**, after the logician Haskell B. Curry. For example, the following definition of a `map` function is curried, because the parameters can be supplied separately¹³ (or together):

```
> fun map f [] = [] | map f (h::t) = (f h):: map f t;
val map = fn : ('a -> 'b) -> 'a list -> 'b list
> val square_list = map square;
val square_list = fn : int list -> int list
> square_list [1,2,3];
val it = [1,4,9] : int list
> map (fn x => x + x) [1,2,3];
val it = [2,4,6] : int list
```

¹³The `map` function as defined here is in fact predefined in ML.

Note that in ML we have the choice of using a tuple to get an “uncurried” version of a function, or two separate parameters to get a curried version:

```
(* uncurried version of a gcd function: *)
> fun gcd (u,v) = if v = 0 then u else gcd (v , u mod v);
val gcd = fn : int * int -> int
(* curried version of a gcd function: *)
> fun gcd u v = if v = 0 then u else gcd v (u mod v);
val gcd = fn : int -> int -> int
```

In fact, in ML, the uncurried version of a function using a tuple is really just a function of a single parameter as well, except that its parameter is a tuple.

We call a language **fully curried** if function definitions are automatically treated as curried, and if all multiparameter built-in functions (such as the arithmetic operators) are curried. ML is not fully curried according to this definition, since all built-in binary operators are defined as taking tuples:

```
> op +;
val it = fn : int * int -> int
> op ::;
val it = fn : 'a * 'a list -> 'a list
> op @;
val it = fn : 'a list * 'a list -> 'a list
```

We could define curried versions of these functions, but they must be used as prefix functions rather than operators:

```
> fun plus x y = x + y;
val plus = fn : int -> int -> int
> plus 2 3;
val it = 5 : int
> val plus2 = plus 2;
val plus2 = fn : int -> int
> plus2 3;
val it = 5 : int
> fun append L1 L2 = L1 @ L2;
val append = fn : 'a list -> 'a list -> 'a list
> val append_to_one = append [1];
val append_to_one = fn : int list -> int list
> append_to_one [2,3];
val it = [1,2,3] : int list
```

3.4 Delayed Evaluation

An important problem that arises in the design and use of functional languages is the distinction between ordinary function applications and special forms. As we have already noted, in a language with an applicative order evaluation rule, such as Scheme and ML, all parameters to user-defined functions are evaluated at the time of a call, even though it may not be necessary to do so—or even wrong to do so.

Typical examples that do not use applicative order evaluation are the Boolean special forms `and` and `or`, as well as the `if` special form. In the case of the `and` special form, the Scheme expression `(and a b)` need not evaluate the parameter `b` if `a` evaluates to false. This is called short-circuit evaluation of Boolean expressions, and it is an example of the usefulness of delayed evaluation. In the case of the `if` special form, it is not a case of just usefulness, but of necessity; for the expression `(if a b c)` in Scheme to have the proper semantics, the evaluation of `b` and `c` must be delayed until the result of `a` is known, and based on that, either `b` or `c` is evaluated, but never both. For this reason, an `if` special form cannot be defined as a standard function in Scheme or ML. It also means that Scheme and ML must distinguish between two kinds of forms, those that use standard evaluation (function applications) and those that do not (the special forms).

A possible argument for restricting function evaluation to applicative order evaluation is that the semantics (and the implementation) are simpler. Consider the case of an expression in which one subexpression may have an undefined value, such as in the Scheme expression `(and (= 1 0) (= 1 (/ 1 0)))` or its C equivalent `(1 == 0) && (1 == 1 / 0)`. In this case, delayed evaluation can lead to a well-defined result, even though subexpressions or parameters may be undefined. Functions with this property are called **nonstrict**, and languages with the property that functions are strict are easier to implement. In essence, strict languages satisfy a strong form of the GIGO principle (garbage in, garbage out), in that they will consistently fail to produce results when given incomplete or malformed input. Strictness also has an important simplifying effect on formal semantics (as discussed in Chapter 12).

Nevertheless, as you have seen, nonstrictness can be a desirable property. Indeed, the argument in favor of nonstrictness is essentially identical to the argument against it. Nonstrict functions can produce results even in the presence of incomplete or malformed input—as long as there is enough information to make the result reasonable. It is interesting to note that the language Algol60 included delayed evaluation in its pass by name parameter passing convention, as will be discussed in detail in Chapter 10. According to pass by name evaluation, a parameter is evaluated only when it is actually used in the code of the called procedure. Thus, the function:

```
function p(x: boolean; y: integer): integer;
begin
  if x then p := 1
  else p := y;
end;
```

will, using pass by name evaluation, return the value 1 when called as `p(true, 1 div 0)`, since `y` is never reached in the code of `p`, and so the value of `y`—the undefined expression `1 div 0`—will never be computed.

In a language that has function values, it is possible to delay the evaluation of a parameter by putting it inside a function “shell” (a function with no parameters). For example, in C, we can achieve the same effect as pass by name in the previous example by writing:

```
typedef int (*IntProc) ();
int divByZero ()
{ return 1 / 0;
}
int p(int x, IntProc y)
{ if (x) return 1;
  else return y();
}
```

and calling `p(1, divByZero)`. (Sometimes such “shell” procedures as `divByZero` are referred to as **pass by name thunks**, or just **thunks**, for somewhat obscure historical reasons.) In Scheme and ML, this process of surrounding parameters with function shells is even easier, since the `lambda` and `fn` function value constructors can be used directly, as in the following Scheme definition:

```
(define (p x y) (if x 1 (y)))
```

which can be called as follows:

```
(p #T (lambda () (/ 1 0)))
```

Note that the code of `p` must change to reflect the function parameter `y`, which must be surrounded by parentheses to force a call to `y`. Otherwise the function itself and not its value will be returned.

Indeed, Scheme has two special forms that do precisely what we have been describing. The special form `delay` delays the evaluation of its arguments and returns an object that can be thought of as a `lambda` “shell,” or **promise** to evaluate its arguments. The corresponding special form `force` causes its parameter, which must be a delayed object, to be evaluated. Thus, the function `p` would be written as:

```
(define (p x y) (if x 1 (force y)))
```

and called as:

```
(p #T (delay (/ 1 0)))
```

Inefficiency results from delayed evaluation when the same delayed expression is repeatedly evaluated. For example, in the delayed version of the square function,

```
(define (delayed-square x) (* (force x) (force x)))
```

the parameter `x` will be evaluated twice in the body of `delayed-square`, which is inefficient if `x` is a complicated expression. In fact, Scheme has an improvement to pass by name built into the `force` function. Scheme uses a **memoization** process, where the value of a delayed object is stored the first time the object is forced, and then subsequent calls to `force` simply return the stored value rather than recomputing it. (This kind of parameter evaluation is sometimes referred to as **pass by need**.)

Pass by name delayed evaluation is helpful in that it allows parameters to remain uncomputed if not needed and permits special forms similar to `if` and `cond` to be defined as ordinary functions. However, pass by name is not able to handle more complex situations where only parts of each parameter are

needed in a computation. Consider the following simple example of a `take` procedure that returns the first `n` items of a list:

```
(define (take n L)
  (if (= n 0) '()
      (cons (car L) (take (- n 1) (cdr L)))))
```

If we write a version in which the computation of `L` is delayed,

```
(define (take n L)
  (if (= n 0) '()
      (cons (car (force L)) (take (- n 1)
                                   (cdr (force L))))))
```

then a call `(take 1 (delay L))` will force the evaluation of the entire list `L`, even though we are interested in the very first element only. This can be disastrously inefficient if `L` happens to be a very long list produced by a list generation procedure such as:

```
(define (intlist m n)
  (if (> m n) '() (cons m (intlist (+ 1 m) n))))
```

Now a call `(take 1 (delay (intlist 2 100)))` will still construct the entire list `(2..100)` before taking the first element to produce the list `(2)`. What is needed is a `delay` in the second parameter to `cons` in `intlist` as well:

```
(define (intlist m n)
  (if (> m n) '() (cons m (delay (intlist (+ 1 m) n)))))
```

so that taking `(cdr (cons m (delay ...)))` returns a delayed object to the recursive call to `take`. Thus, we have the following sequence of events in this computation, where each delayed object is represented by the computation it “promises” to perform enclosed in quotes:

1. The call `(take 1 (delay (intlist 2 100)))` causes the delayed object `(intlist 2 100)` to be passed as `L` to:

```
(cons (car (force L)) (take (- 1 1) (cdr (force L))))
```

2. The first call to `(force L)` in the `cons` causes `L` to be evaluated to:

```
(cons 2 ((delay (intlist (+ 1 2) 100))))
```

which causes the construction of the list with head 2 and tail the delayed object `(intlist (+ 1 2) 100)`.

3. The `car` of `(force L)` returns 2 from the `cons`, leaving the following expression to be evaluated:

```
(cons 2 (take (- 1 1) (cdr (force L))))
```

4. The call to `(take (- 1 1) (cdr (force L)))` causes `(- 1 1)` to be evaluated to 0, and the `cdr` of `(force L)` to be evaluated to the delayed object `(intlist (+ 1 2) 100)` as constructed in Step 2. The expression

(take 0 (intlist (+ 1 2) 100)) is then evaluated and returns the empty list '() without forcing the evaluation of (intlist (+ 1 2) 100).

5. The result of (cons 2 '()) is finally evaluated, returning the list (2). Thus, the rest of the integer list is never computed.

The scenario we have just described, together with memoization, is called **lazy evaluation**. This mechanism of evaluating expressions can be achieved in a functional language without explicit calls to `delay` and `force` by requiring the runtime environment to adhere to the following rules:

1. All arguments to user-defined functions are delayed.
2. All bindings of local names in `let` and `letrec` expressions are delayed.
3. All arguments to constructor functions (such as `cons`) are delayed.
4. All arguments to other predefined functions, such as the arithmetic functions “1,” “*,” and so on, are forced.
5. All function-valued arguments are forced.
6. All conditions in selection forms such as `if` and `cond` are forced.

According to these rules, operations on long lists can only compute as much of the list as is necessary. Lazy evaluation also makes it possible to include potentially infinite lists in languages, because only a part of the list would ever be computed at any one time. Lists that obey lazy evaluation rules are often called **streams**. This name is suggestive of the infinite nature of lists. A stream can be thought of as a partially computed list whose remaining elements can continue to be computed up to any desired number. Streams are an important mechanism in functional programming. To eliminate side effects completely, input and output must be introduced into functional languages as streams. Note that both Scheme and ML have ad hoc stream constructions (besides the manual `delay` and `force` procedures mentioned earlier).¹⁴

The primary example of a functional language with lazy evaluation is Haskell (again, named after Haskell B. Curry). Lazy evaluation supports a style of functional programming known as **generator-filter programming**. The programmer can separate a computation into procedures that generate streams and other procedures that take streams as arguments, without worrying about the efficiency of each step. Procedures that generate streams are called **generators**, and procedures that modify streams are called **filters**. For example, the `intlist` procedure in a previous Scheme example is a generator, and the `take` procedure is a filter.

A famous problem in functional programming that requires generator-filter programming is the **same-fringe** problem for lists. Two lists have the same fringe if they contain the same non-null atoms in the same order, or, to put it another way, when they are written as trees, their non-null leaves taken in left-to-right order are the same. For example, the lists ((2 (3)) 4) and (2 (3 4 ())) have the same fringe. To determine whether two lists have the same fringe, we must **flatten** them to just lists of their atoms:

¹⁴Haskell has replaced I/O streams with a more general construct called a monad, which we also do not study here.

```
(define (flatten L)
  (cond ((null? L) '())
        ((list? L) (append (flatten (car L)) (flatten (cdr L))))
        (else (cons L '())))))
```

In the case of both lists in the previous paragraph, `flatten` returns the list `(2 3 4)`. The `flatten` function can be viewed as a filter, which then can be used as the input to the `samefringe` procedure, as follows:

```
(define (samefringe? L M) (equal? (flatten L)
                                   (flatten M)))
```

The problem with this computation is that, under the usual Scheme evaluation rule, `flatten` will produce complete lists of the elements of `L` and `M`, even if `L` and `M` differ already in their first fringe elements. Lazy evaluation, on the other hand, will compute only enough of the flattened lists as necessary before their elements disagree. Only when the lists actually do have the same fringe must the entire lists be processed.

A similar situation arises when using the sieve of Eratosthenes algorithm to compute prime numbers. In this method, a list of consecutive integers from 2 is generated, and each successive remaining number in the list is used to cancel out its multiples in the list. Thus, beginning with the list `(2 3 4 5 6 7 8 9 10 11)`, we first cancel multiples of 2, leaving the list `(2 3 5 7 9 11)`. We then cancel the multiples of 3, giving `(2 3 5 7 11)`, and now we cancel all remaining multiples of 5, 7, and 11 (of which there are none) to obtain the list of primes from 2 to 11 as `(2 3 5 7 11)`. Each cancellation step can be viewed as a filter on the list of the previous cancellation step. We will see in the next section how both the sieve of Eratosthenes problem and the fringe comparison problem can be solved in Haskell.

Why don't all functional languages use delayed evaluation? For one thing, it complicates the semantics of the language. In practical terms, this translates into an increased complexity in the runtime environment that must maintain the evaluation rules 1–6 listed earlier. Also, because of the interleaving of delays and forces, it is difficult to write programs with desired side effects, in particular, programs with variables that change as computation proceeds. In a way, this is similar to the synchronization problem for shared memory in parallel processing (studied in Chapter 13). Indeed, delayed evaluation has been described as a form of parallelism, with `delay` as a form of process suspension, and `force` a kind of process continuation. The principal point is that side effects, in particular assignment, do not mix well with lazy evaluation. This is in part the reason that Haskell is purely functional, with no variables or assignment, and why Scheme and ML are strict languages, with some ad hoc stream and delayed evaluation facilities.

3.5 Haskell—A Fully Curried Lazy Language with Overloading

Haskell is a pure functional language whose development began in the late 1980s, primarily at Yale University and the University of Glasgow. Its current standard is Haskell98, which we use in our discussion here. Haskell builds on and extends a series of purely functional lazy languages developed in the late 1970s and 1980s by David A. Turner, culminating in the Miranda programming language. Haskell also

contains a number of novel features, including function overloading and a mechanism called **monads** for dealing with side effects such as I/O (always a problem for pure functional languages). For more on the monad or I/O features, see the Notes and References for this chapter. In the following discussion, we focus on Haskell's lazy evaluation and overloading.

3.5.1 Elements of Haskell

Haskell's syntax is very similar to that of ML, with some notable differences. Haskell reduces the amount of syntax to an absolute minimum using internal program clues, including a **layout rule** that uses indentation and line formatting to resolve ambiguities. As a result, it is rare for a Haskell program to require a semicolon or bracketing of any kind. Indeed, no special syntax for definitions is required, and no vertical bar is used for functions defined using pattern matching. Here are a few examples of function definitions in Haskell:

```
fact 0 = 1
fact n = n * fact (n - 1)
square x = x * x
gcd1 u v = if v == 0 then u else gcd1 v (mod u v)
reverse1 [] = []
reverse1 (h:t) = reverse1 t ++ [h]
```

In these definitions we note the following differences between Haskell and ML. First, Haskell does not allow the redefinition of functions that are already predefined. (However, it is easy to change what is actually predefined.) Thus, we append a 1 to the functions `reverse` and `gcd` in the preceding code, since they are already predefined. Next, we note that the `cons` operator for constructing lists is written as a single colon. Types, on the other hand, are given using a double colon, which is exactly the reverse of ML. Pattern matching, as already mentioned, does not require the use of the `.` symbol. Finally, list concatenation in Haskell is given by the `++` operator.

Haskell is a fully curried language. All predefined binary operators are curried. In a construct called a **section** in Haskell, a binary operator can be partially applied to either argument using parentheses. Thus,

```
plus2 = (2 +)
```

defines a function that adds 2 to its argument on the left, while:

```
times3 = (* 3)
```

defines a function that multiplies 3 times its argument on the right:

```
> plus2 3
5
> times3 4
12
```

Infix functions can also be turned into prefix functions by surrounding them with parentheses. No `op` keyword is required. For example:

```
> (+) 2 3
5
> (*) 4 5
20
```

Haskell has anonymous functions or `lambda` forms, with the backslash representing the `lambda`:

```
> (\x -> x * x ) 3
9
```

3.5.2 Higher-Order Functions and List Comprehensions

Haskell includes many predefined higher-order functions, such as `map`. These are all in curried form (as noted above), so they can be partially applied to construct further higher-order functions. For example, the `map` function has type:

```
(a -> b) -> [a] -> [b]
```

A `square_list` function can be defined as follows, leaving off the list parameter by currying:

```
square_list = map (\x -> x * x)
```

As another example, the function composition operator in Haskell is denoted by the dot (`.`):

```
> ((*3) . (2+)) 5
21
```

Haskell, like ML, has built-in lists and tuples, as well as type synonyms and user-defined polymorphic types:

```
type ListFn a = [a] -> [a]
type Salary = Float
type Pair a b = (a,b)
data BST a = Nil | Node a (BST a) (BST a)
```

Note that type variables in Haskell (such as `a` and `b` in the preceding definitions) are written without the quote of ML, and are also written after the data type name, rather than before. The keyword `data` replaces the ML keyword `datatype`, and new user-defined types are written in a slightly different syntax without the `of` keyword. Type and constructor names in Haskell are also required to be uppercase, while function and value names must be lowercase.

Functions on new data types can use data constructors as patterns, as in ML:

```
flatten:: BST a -> [a]
flatten Nil = []
flatten (Node val left right) =
    (flatten left) ++ [val] ++ (flatten right)
```

Note that the definition is preceded by the type of the `flatten` function. The type specifier is permitted, although not required. A type specifier can be used to resolve ambiguities or restrict the type of the function beyond what Hindley-Milner type checking would infer. The use of this option is similar to the way ML allows the types of the parameters and returned types to be specified within the definition itself, something not allowed in Haskell98.

Haskell also has a special notation for operations that are applied over lists, called **list comprehensions**. List comprehensions are designed to look like set definitions in mathematics. For example, if we wanted to square a list of integers, we could write the following:

```
square_list lis = [ x * x | x <- lis]
```

This is really just syntactic sugar for `map (\x -> x * x) lis`, but it can make programming with lists even easier and more transparent. The notation can also be extended to include Boolean conditions, such as:

```
square_list_positive lis = [ x * x | x <- lis, x > 0]
```

which is also syntactic sugar for `map (\x -> x * x) (filter (>0) lis)`, where `filter` is a predefined list function defined as follows:

```
filter:: (a -> Bool) -> [a] -> [a]
filter pred [] = []
filter pred (h:t) =
    if pred h then h : filter pred t else filter pred t
```

As another example of the use of list comprehensions, a highly compact definition for quicksort can be given (using the first element of a list as the pivot):

```
qsort [] = []
qsort (h:t) = qsort [x | x <- t, x <= h] ++
    [h] ++ qsort [x | x <- t, x > h]
```

3.5.3 Lazy Evaluation and Infinite Lists

Because Haskell is a lazy language, no value is computed unless it is actually needed for the computation of a program. Thus, given the definition:

```
f x = 2
```

any call to `f` will never evaluate its argument, since it is not needed for the result. Similarly, the following function will behave exactly like the built-in `if-then-else` expression:

```
my_if x y z = if x then y else z
```

Lazy evaluation also means that lists in Haskell are the same as streams, and are capable of being potentially infinite. Thus, we could define an infinite list of ones as follows, without danger of getting into an immediate infinite loop:

```
ones = 1 : ones
```

Haskell has, in fact, several shorthand notations for infinite lists. For example, the notation `[n..]` stands for the list of integers beginning with `n`. This same list can also be written as `[n, n+1..]`, and the list of ones can also be defined as `[1,1..]`. As a final example of this notation, the list of even positive integers can be written as `[2,4..]`.

Given a potentially infinite list, it is of course impossible to display or process the entire list. Thus, functions that partially compute the list become useful. Two such functions are `take` and `drop`; `take` extracts the first `n` items from a list, and `drop` discards the first `n` items (the underscore is the wildcard pattern, as in ML):

```
take 0 _ = []
take _ [] = []
take n (h:t) = h : take (n - 1) t
drop 0 lis = lis
drop _ [] = []
drop n (_:t) = drop (n - 1) t
```

For example, `take 5 (drop 4 [2..])` gives the result `[6,7,8,9,10]`.

Using lazy evaluation, it is possible to get an extremely compact representation of the Sieve of Eratosthenes introduced earlier in the discussion on ML:

```
sieve (p : lis) = p : sieve [n | n <- lis , mod n p /= 0]
primes = sieve [2..]
```

With this definition of the (infinite) list of primes, `take` can be used to compute any desired number of primes:

```
> take 100 primes
[2,3,5,7,11,13,17,19,23,29,31,37,41,43,47,53,59,61,67,71,
73,79,83,89,97,101,103,107,109,113,127,131,137,139,149,
151,157,163,167,173,179,181,191,193,197,199,211,223,
227,229,233,239,241,251,257,263,269,271,277,281,283,
293,307,311,313,317,331,337,347,349,353,359,367,373,
379,383,389,397,401,409,419,421,431,433,439,443,449,
457,461,463,467,479,487,491,499,503,509,521,523,541]
```

3.5.4 Type Classes and Overloaded Functions

The last topic we discuss in this short overview of Haskell is its unique approach to overloading. Recall ML functions cannot be overloaded, and this either led to unresolved ambiguities or strong assumptions that ML had to make to avoid them. In particular, we could not define an arithmetic function such as `square` that would work for both integers and real numbers. Haskell has no such problem. Indeed, we can define a `square` function as usual:

```
square x = x * x
```

and then use it on any numeric value:

```
> square 2
4
> square 3.5
12.25
```

The type of `square` gives us a hint as to how Haskell does this:

```
square :: Num a => a -> a
```

This type is called a **qualified type**. It says essentially that `square` can be applied to any type `a` as long as `a` is a `Num`. But what is a `Num`? It is not a type, but a **type class**. That is, it is a set of types that all define certain functions. Here is a possible, though incomplete, definition of the `Num` type class:

```
class Num a where
  (+), (-), (*) :: a -> a -> a
  negate      :: a -> a
  abs         :: a -> a
```

A type class definition states the names and types, also called **signatures**, of the functions that every type belonging to it must define. Thus, type classes are a little like Java interfaces. By itself, of course, this definition does nothing. For any types to belong to this class, we must give an **instance definition**, in which actual working definitions for each of the required functions are provided. For example, in the standard library, the data types `Int`, `Float`, and `Double` are all instances of `Num`. A (partial) instance definition for `Int` in the standard library looks as follows:

```
instance Num Int where
  (+)      = primPlusInt
  (-)      = primMinusInt
  negate   = primNegInt
  (*)      = primMulInt
  abs      = absReal
```

The functions to the right of the equal signs are all built-in definitions that are hidden inside the standard library modules.

Haskell determines the type of the `square` function by looking for the symbol `*` in a class definition (because the body of `square` depends on the availability of `*`) and applying the appropriate qualification. Then, the `square` function can be applied to any value whose type is an instance of the `Num` class.

There is, however, more structure to type classes than this simple description suggests. In the case of `Int`, for example, the `Num` class does not describe all of the functions that need to be available, such as equality tests and order functions like `<`. Nor does the `Int` instance include the functions `div` and `mod`. The `Num` class above does not even require any division operation. Additional type classes do require these functions. The class `Eq` requires the equality operator `==`, the class `Ord` requires the less-than operator `<`, and the class `Integral` requires the `div` and `mod` operations. To provide full capability, `Int` should be defined to be an instance of all of these type classes. To accomplish this, many of the type classes themselves are defined to be part of other type classes, so that any instances are forced to be instances of these other classes as well. This dependency of type classes on other type classes is called **type class inheritance**. Type inheritance relies upon a hierarchy of type classes. Two type classes at the base of this hierarchy are the `Eq` and `Show` classes. The class `Show` is used to establish the existence of a `show` function, which is used to display values of member types. All predefined Haskell types are instances of the `Show` class. The class `Eq` establishes the ability of two values of a member type to be compared using the `==` operator. Most types are instances of `Eq`. However, function types are not, because the equality of functions cannot be defined in a way that would provide useful information.

Here is a complete definition of the `Eq` class from the Haskell standard library:

```
class Eq a where
    (==), (/=) :: a -> a -> Bool
    x == y      = not (x/=y)
    x /= y      = not (x==y)
```

This class definition shows an additional feature: the use of default definitions for the required functions. Such default definitions are typically based on other required functions. In the preceding code, the default definition of `/=` (not equal) depends on the definition of `==` being available, and the default definition of `==` depends on the definition of `/=` being available. This apparent circularity means that, for a type to be declared an instance of `Eq`, only one of the two required functions needs to be defined, with the other implicitly defined by default. For example, `Int` is defined to be an instance of `Eq` as follows:

```
instance Eq Int where (==) = primEqInt
```

To establish the inheritance requirements for each type class, Haskell uses a notation similar to type qualification. For example, the definition of the `Num` class in the Haskell standard library begins as follows:

```
class (Eq a, Show a) => Num a where ...
```

And the definition of the `Ord` class is as follows (with the default function definitions removed):

```
class (Eq a) => Ord a where
  compare      :: a -> a -> Ordering
  (<), (<=), (>=), (>) :: a -> a -> Bool
  max, min     :: a -> a -> a
```

The numeric class hierarchy of Haskell is shown in Figure 3.10, along with sample functions that are required by some of the type classes.

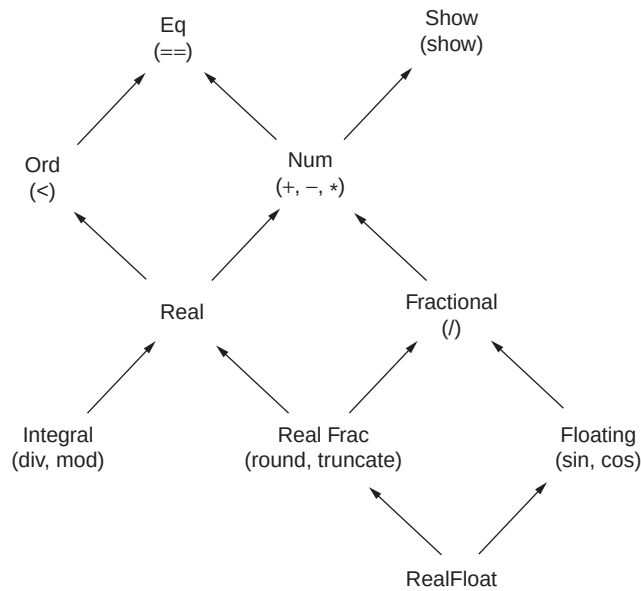


Figure 3.10 The numeric type class hierarchy in Haskell, with sample functions required by some of the classes in parentheses

The following is a final example of the use of type classes in Haskell. When a program defines a new data type, such as the `BST` type defined previously,

```
data BST a = Nil | Node a (BST a) (BST a)
```

then such trees cannot in general either be displayed or compared for equality, since we have not defined `BST` to be an instance of the `Show` or `Eq` type classes. For example, if we define:

```
tree = Node "horse"
      (Node "cow" Nil (Node "dog" Nil Nil))
      (Node "zebra" (Node "yak" Nil Nil) Nil)
```

then a Haskell interpreter will respond as follows when we try to display or compare `tree`:¹⁵

```
> tree
ERROR: Cannot find "show" function for:
*** Expression : tree
*** Of type    : Tree [Char]
> tree == tree
ERROR: Illegal Haskell 98 class constraint in inferred type
*** Expression : tree == tree
*** Type       : Eq (BST [Char]) => Bool
```

In order to make a user-defined data type useful, we typically must define it to be an instance of `Eq` and `Show`. Note that `Show` requires a function `show` that converts the data type in question into a `String` so that it can be displayed. For example:

```
instance Eq a => Eq (BST a)
  where
    Nil == Nil = True
    Node x a b == Node y c d = x == y && a == c && b == d
    _ == _ = False

instance Show a => Show (BST a) where show = ...
```

Note in particular how the definition of equality for a BST depends on the membership of the type parameter `a` in `Eq`, because otherwise the equality of the stored data cannot be tested.

It is often necessary to define a new data type to be a member of certain classes like `Eq`, `Show`, and `Ord`, so Haskell offers a way to automatically generate these definitions. Haskell simply assumes that the “natural” definitions are desired. A data type is shown by printing it out as a string, and equality is defined component-wise. To accomplish this, a deriving clause is added to the data definition:

```
data BST a = Nil | Node a (BST a) (BST a)
  deriving (Show, Eq)
```

With this definition of `BST`, Haskell has no trouble displaying a BST or comparing two BSTs for equality:

```
> tree
Node "horse" (Node "cow" Nil (Node "dog" Nil Nil))
  (Node "zebra" (Node "yak" Nil Nil) Nil)
> tree == tree
True
```

¹⁵This is the output from the HUGS Haskell interpreter—see the Notes and References.

3.6 The Mathematics of Functional Programming: Lambda Calculus

Lambda calculus was invented by Alonzo Church in the 1930s as a mathematical formalism for expressing computation by functions, similar to the way a Turing machine is a formalism for expressing computation by a computer. In fact, lambda calculus can be used as a model for (purely) functional programming languages in much the same way that Turing machines can be used as models for imperative programming languages. Thus, it is an important result for functional programming that lambda calculus as a description of computation is equivalent to Turing machines. This implies the result that a purely functional programming language, with no variables and no assignment, and with an `if-then-else` construct and recursion, is Turing complete. This means that computation performed by a Turing machine can be described by such a language.

Lambda calculus provides a particularly simple and clear view of computation. It is useful for anyone interested in functional programming to have at least some knowledge of lambda calculus, because many functional languages, including Lisp, ML, and Haskell, were based on lambda calculus. Therefore, we offer this section as a basic introduction to the subject. For those with more than a superficial interest in functional programming, we recommend consulting one of the texts listed at the end of the chapter.

The essential construct of lambda calculus is the **lambda abstraction**:

$$(\lambda x . 1 + x)$$

This can be interpreted exactly as the lambda expression:

$$(\text{lambda } (x) (+ 1 x))$$

in Scheme, namely, as representing an unnamed function of the parameter x that adds 1 to x . Note that, like Scheme, expressions such as $(1 + x)$ are written in prefix form.

The basic operation of lambda calculus is the **application** of expressions such as the lambda abstraction. The expression

$$(\lambda x . + 1 x) 2$$

represents the application of the function that adds 1 to x to the constant 2. Lambda calculus expresses this by providing a **reduction rule** that permits 2 to be substituted for x in the lambda (and removing the lambda), yielding the desired value:

$$(\lambda x . + 1 x) 2 \Rightarrow (+ 1 2) \Rightarrow 3$$

The syntax for lambda calculus is as follows:

$$\begin{aligned} &exp \rightarrow constant \\ &\quad | variable \end{aligned}$$

$$| (exp\ exp)$$

$$| (\lambda\ variable\ .\ exp)$$

Constants in this grammar are numbers like 0 or 1 and certain predefined functions like 1 and *. Variables are names like x and y . The third rule for expressions represents function application ($f\ x$) as noted earlier. The fourth rule gives lambda abstractions. Note that only functions of a single variable are allowed, but since lambda expressions provide for higher-order functions, this is not a significant restriction, as the notion of currying, discussed earlier in this chapter, allows for functions of several variables to be interpreted simply as functions of a single variable returning functions of the remaining variables. Thus, lambda calculus as defined here is fully curried.

Variables in lambda calculus are not like variables in a programming language—they do not occupy memory, because lambda calculus has no concept of memory. Lambda calculus variables correspond instead to function parameters as in purely functional programming. The set of constants and the set of variables are not specified by the grammar. Thus, it is more correct to speak of many **lambda calculi**. Each specification of the set of constants and the set of variables describes a particular lambda calculus. Lambda calculus without constants is called pure lambda calculus.

Parentheses are included in the rules for function application and lambda abstraction to avoid ambiguity, but by convention may be left out, in case no ambiguity results. Thus, according to our grammar, we should have written $(\lambda x. ((+ 1) x))$ for $(\lambda x. + 1\ x)$ and $((\lambda x. ((+ 1) x) 2)$ for $(\lambda x. + 1\ x) 2$. However, no ambiguity results from the way they were written.

The variable x in the expression $(\lambda x.E)$ is said to be **bound** by the lambda. The scope of the binding is the expression E . An occurrence of a variable outside the scope of any binding of it by a lambda is a **free occurrence**. An occurrence that is not free is a **bound** occurrence. Thus, in the expression $(\lambda x. E)$, all occurrences of x in E are bound. For example, in the expression

$$(\lambda x. + y\ x)$$

x is bound and y is free. If we try to apply the function specified by this lambda abstraction, we substitute for x , but we have no way of determining the value of y :

$$(\lambda x. + y\ x) 2 \Rightarrow (+ y\ 2)$$

The variable y is like a nonlocal reference in the function—its value must be specified by an external environment.

Different occurrences of a variable can be bound by different lambdas, and some occurrences of a variable may be bound, while others are free. In the following expression:

$$(\lambda x. + ((\lambda y. ((\lambda x. * x\ y) 2)) x) y)$$

the occurrence of x after the $*$ is bound by a different lambda than the outer x . Also, the first occurrence of y is bound, while the second occurrence is free. In fact, applying the functions defined by the lambdas gives the following reduction:

$$\begin{aligned} (\lambda x. + ((\lambda y. ((\lambda x. * x y) 2)) x) y) &\Rightarrow \\ (\lambda x. + ((\lambda y. (* 2 y)) x) y) &\Rightarrow \\ (\lambda x. + (* 2 x) y) \end{aligned}$$

When reducing expressions that contain multiple bindings with the same name, care must be taken not to confuse the scopes of the bindings—sometimes renaming is necessary (discussed shortly).

We can view lambda calculus as modeling functional programming by considering a lambda abstraction as a function definition and the juxtaposition of two expressions as function application. As a model, however, lambda calculus is extremely general. For example, nothing prevents us in lambda calculus from applying a function to itself or even defining a function that takes a function parameter and applies it to itself: $(x x)$ and $(\lambda x. x x)$ are legal expressions in lambda calculus. A more restrictive form of lambda calculus, called the **typed lambda calculus**, includes the notion of data type from programming languages, thus reducing the set of expressions that are allowed. We will not consider the typed lambda calculus further.

Since lambda calculus is so general, however, very precise rules must be given for transforming expressions, such as substituting values for bound variables. These rules have historical names, such as “alpha-conversion” and “beta-conversion.” We will give a short overview to show the kinds of operations that can occur on lambda expressions.

The primary method for transforming lambda expressions is by **substitution**, or **function application**. We have seen a number of examples of this already: $((\lambda x. + x 1) 2)$ is equivalent to $(+ 2 1)$, substituting 2 for x and eliminating the lambda. This is exactly like a call in a programming language to the function defined by the lambda abstraction, with 2 substituted for x as a value parameter. Historically, this process has been called **beta-reduction** in lambda calculus. One can also view this process in reverse, where $(+ 2 1)$ becomes $((\lambda x. + x 1) 2)$, in which case it is called **beta-abstraction**. **Beta-conversion** refers to either beta-reduction or beta-abstraction and establishes the equivalence of the two expressions. Formally, beta-conversion says that $((\lambda x. E) F)$ is equivalent to $E[F/x]$, where $E[F/x]$ is E with all free occurrences of x in E replaced by F .

Care must be taken in beta-conversion when F contains variable names that occur in E . Consider the expression:

$$((\lambda x. (\lambda y. + x y)) y)$$

The first occurrence of y is bound, while the second is free. If we were to replace x by y blindly, we would get the incorrect reduction $(\lambda y. + y y)$. This is called the **name capture** problem. What we have to do is change the name of y in the inner lambda abstraction so that the free occurrence will not conflict, as follows:

$$((\lambda x. (\lambda y. + x y)) y) \Rightarrow ((\lambda x. (\lambda z. + x z)) y) \Rightarrow (\lambda z. + y z)$$

Name change is another available conversion process in lambda calculus, called **alpha-conversion**: $(\lambda x.E)$ is equivalent to $(\lambda y.E[y/x])$, where as before, $E[y/x]$ stands for the expression E with all free occurrences of x replaced by y . (Note that if y is a variable that already occurs in E there can be trouble similar to the substitution problem just discussed.)

Finally, there is a conversion that allows for the elimination of “redundant” lambda abstractions. This conversion is called **eta-conversion**. A simple example is the expression $(\lambda x. (+ 1 x))$. Since 1 is curried, the expression $(+ 1)$ is a function of one argument, and the expression $(+ 1 x)$ can be viewed as the application of the function $+ 1$ to x . Thus, $(\lambda x. (+ 1 x))$ is equivalent to the function $(+ 1)$ without the lambda abstraction. In general, $(\lambda x. (E x))$ is equivalent to E by eta-conversion, as long as E contains no free occurrences of x . A further example of eta-conversion is:

$$(\lambda x. (\lambda y. (+ x y))) \Rightarrow (\lambda x. (+ x)) \Rightarrow 1$$

which is to say that the first expression is just another notation for the 1 function.

It is worth noting that eta-reduction is helpful in simplifying curried definitions in functional languages. For example, the ML definition

```
fun square_list lis = map (fn x => x * x) lis ;
```

can be simplified by eta-reduction to

```
val square_list = map (fn x => x * x) ;
```

The order in which beta-reductions are applied to a lambda expression can have an effect on the final result obtained, just as different evaluation orders can make a difference in programming languages. In particular, it is possible to distinguish applicative order evaluation (or pass by value) from normal order evaluation (or pass by name). For example, in the following expression:

$$((\lambda x. * x x) (+ 2 3))$$

we could either use applicative order, replacing $(+ 2 3)$ by its value and then applying beta-reduction, as in

$$((\lambda x. * x x) (+ 2 3)) \Rightarrow ((\lambda x. * x x) 5) \Rightarrow (* 5 5) \Rightarrow 25$$

or we could apply beta-reduction first and then evaluate, giving normal order evaluation:

$$((\lambda x. * x x) (+ 2 3)) \Rightarrow (* (+ 2 3) (+ 2 3)) \Rightarrow (* 5 5) \Rightarrow 25$$

Normal order evaluation is, as we have noted several times, a kind of **delayed** evaluation, since the evaluation of expressions is done only after substitution.

Normal order evaluation can have a different result than applicative order, as we have also previously noted. A striking example is when parameter evaluation gives an undefined result. If the value of an expression does not depend on the value of the parameter, then normal order will still compute the correct value, while applicative order will also give an undefined result.

For example, consider the lambda expression $((\lambda x. x x) (\lambda x. x x))$. Beta-reduction on this expression results in the same expression all over again. Thus, beta-reduction goes into an “infinite loop,” attempting to reduce this expression. If we use this expression as a parameter to a constant expression, as in:

$$((\lambda y. 2) ((\lambda x. x x) (\lambda x. x x)))$$

then using applicative order, we get an undefined result, while using normal order we get the value 2, since it does not depend on the value of the parameter y .

Functions that can return a value even when parameters are undefined are said to be **nonstrict**, while functions that are always undefined when a parameter is undefined are **strict**. In lambda calculus notation, the symbol $\perp$ (pronounced “bottom”) represents an undefined value, and a function f is strict if it is always true that $(f \perp) = \perp$. Thus, applicative order evaluation is strict, while normal order evaluation is nonstrict.

Normal order reduction is significant in lambda calculus in that many expressions can be reduced, using normal order, to a unique normal form that cannot be reduced further. This is a consequence of the famous **Church-Rosser theorem**, which states that reduction sequences are essentially independent of the order in which they are performed. The equivalence of two expressions can then be determined by reducing them using normal order and comparing their normal forms, if they exist. This has practical applications for translators of functional languages, too, since a translator can use normal forms to replace one function by an equivalent but more efficient function.

Finally, in this brief overview of lambda calculus, we wish to show how lambda calculus can model recursive function definitions in exactly the way we have treated them in the previous section. Consider the following definition of the factorial function:

$$\text{fact} = (\lambda n. (\text{if } (= n 0) 1 (* n (\text{fact } (- n 1)))))$$

We remove the recursion by creating a new lambda abstraction as follows:

$$\text{fact} = (\lambda F. \lambda n. (\text{if } (= n 0) 1 (* n (F (- n 1))))) \text{ fact}$$

In other words, the right-hand side can be viewed as a lambda abstraction, which, when applied to fact , gives fact back again.

If we write this abstraction as

$$H = (\lambda F. \lambda n. (\text{if } (= n 0) 1 (* n (F (- n 1)))))$$

then the equation becomes

$$\text{fact} = H \text{ fact}$$

or that fact is a **fixed point** of H . We could construct a least-fixed-point solution by building up a set representation for the function. However, in the lambda calculus, lambda expressions are primitive; that is, we cannot use sets to model them. Thus, to define a recursive function fact in the lambda calculus, we

need a function Y for constructing a fixed point of the lambda expression H . Y needs, therefore, to have the property that $Y H = \text{fact}$ or $Y H = H (Y H)$. Such a Y can indeed be defined by the following lambda expression:

$$Y = (\lambda h. (\lambda x. h (x x)) (\lambda x. h (x x))).$$

Y is called a **fixed-point combinator**, a combinator being any lambda expression that contains only bound variables. Such a Y actually constructs a solution that is in some sense the “smallest.” Thus, one can refer to the **least-fixed-point semantics** of recursive functions in lambda calculus, as well as in the ordinary set-based theory of functions.

Exercises

- 3.1** The following C function computes the power ab , where a is a floating-point number and b is a (non-negative) integer:

```
double power(double a, int b){
    int i;
    double temp = 1.0;
    for (i = 1; i <= b; i++) temp *= a;
    return temp;
}
```

- (a) Rewrite this procedure in functional form.
 (b) Rewrite your answer to (a) using an accumulating parameter to make it tail recursive.
- 3.2** A function that returns the maximum value of a 0-ended list of integers input from a user is given by the following C function:

```
int readMax(){
    int max, x;
    scanf("%d",&x);
    if (x != 0){
        max = x;
        while (x != 0){
            scanf("%d",&x);
            if (x > max) max = x;
        }
        return max;
    }
}
```

Rewrite this procedure as much as possible in functional style using tail recursion (the `scanf` procedure prevents a complete removal of variables).

- 3.3** Recursive sorts are easier to write in functional style than others. Two recursive sorts are Quicksort and Mergesort. Write functional versions of **(a)** Quicksort; **(b)** Mergesort in an imperative language of your choice (e.g., C, Ada, C++, Python), using an array of integers as your basic data structure.
- 3.4** It is possible to write nonrecursive programs that implement recursive algorithms using a stack, but at the cost of extra source code complexity (a necessary overhead of using a nonrecursive language like FORTRAN). Write nonrecursive versions of **(a)** Quicksort; **(b)** Mergesort. Discuss the difficulties of the added complexity. **(c)** Which of the two sorts is easier to implement nonrecursively? Why?
- 3.5** Which of the following functions are referentially transparent? Explain your reasoning.
- (a)** The factorial function
 - (b)** A function that returns a number from the keyboard
 - (c)** A function `p` that counts the number of times it is called
- 3.6** Is a function that has no side effects referentially transparent? Explain.
- 3.7** The binomial coefficients are a frequent computational task in computer science. They are defined as follows for $n \geq 0$, $0 \leq k \leq n$ ($!$ is factorial and $0! = 1$):

$$B(n, k) = \frac{n!}{(n - k)! k!}$$

- (a)** Write a procedure using a loop to compute $B(n, k)$. Test your program on $B(10, 5)$.
- (b)** Use the following recurrence and the fact that $B(n, 0) = 1$ and $B(n, n) = 1$ to write a functional procedure to compute $B(n, k)$:

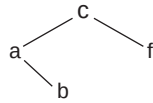
$$B(n, k) = B(n - 1, k - 1) + B(n - 1, k)$$

Test your program on $B(10, 5)$.

- (c)** Can you write a more efficient program than your program of **(b)** and still preserve the functional style? (*Hint*: Think of pass by need memoization as discussed in Section 3.5.)
- 3.8** Complete the following functional programming exercises using Scheme, ML, or Haskell:
- (a)** Write a tail-recursive function to compute the length of an arbitrary list.
 - (b)** Write a function that computes the maximum and minimum of a list of integers.
 - (c)** Write a function that collects integers from the user until a 0 is encountered and returns them in a list in the order they were input (Scheme or ML only).
 - (d)** Use the functions you wrote for exercises (b) and (c) to write a program to input a 0-ended list of integers, print the list in the order entered, and print the maximum and minimum of the list. For this exercise, use only Scheme or ML.
 - (e)** Write Quicksort for a list of integers (Scheme or ML only).
 - (f)** Write Mergesort for a list of integers.
 - (g)** A Pythagorean triple is a tuple of integers (x, y, z) such that $x^2 + y^2 = z^2$. Write a function with a parameter n to print all Pythagorean triples such that $1 \leq x \leq y \leq z \leq n$.
 - (h)** Write a higher-order function `twice` that takes as a parameter a function of one argument and returns a function that represents the application of that function to its argument twice. Given the usual definition of the square function, what function is `(twice (twice square))`?

- (i) Write a higher-order function `inc_n` that takes an integer `n` as a parameter and returns an n th increment function, which increments its parameter by `n`. Thus, in Scheme syntax, `((inc_n 3) 2) = 5` and `((inc_n -2) 3) = 1`.
- 3.9** Suppose that you want to use a list data structure to implement a `Set` data type. Write `insert` and member operations in (a) Scheme; (b) ML; (c) Haskell. Make sure that in ML and Haskell your `Set` data type is distinct from a list. (d) In Haskell, define your `Set` type to be an instance of classes `Eq` and `Show`.
- 3.10** Given the definition of a binary search tree in Haskell as given in the text:
- ```
data BST a = Nil | Node a (BST a) (BST a)
```
- write a `show` function (`show :: (BST a) -> String`) as it would be generated by a Haskell deriving `Show` clause.
- 3.11** Many functions discussed in this chapter are not completely tail recursive, but are almost tail recursive. In these functions, the recursive call comes just before an arithmetic operation, which is the last operation in the function. For example, factorial and length of a list are almost tail recursive in this sense. Describe a general pattern for turning an almost tail-recursive function into a loop, in the same way that a tail-recursive function can be so transformed, as described in Section 3.1. Can a translator recognize almost tail recursion as easily as tail recursion? How might it do so?
- 3.12** Draw box and pointer diagrams for the following Scheme lists:
- ```
((((a)))
 (1 (2 (3 4)) (5))
 ((a ( )) ((c) (d) b) e))
```
- 3.13** Use the box and pointer diagrams of the last exercise to compute the following for each Scheme list for which they make sense:
- ```
(car (car L))
(car (cdr L))
(car (cdr (cdr (cdr L))))
(cdr (car (cdr (cdr L))))
```
- 3.14** Represent the following elements as `car/cdr` expressions of the lists of Exercise 3.12:
- of the first list
  - of the second list
  - of the third list
- 3.15** Scheme's basic data structure is actually a little more general than the lists described in this chapter. Indeed, since the `car` of a list can be a list or an atom, it is also possible for the `cdr` to be an atom as well as a list. In the case when the `cdr` is not a list, the resulting structure is called a **pair** or **S-expression** instead of a list and is written `(a . b)`, where `a` is the `car` and `b` is the `cdr`. Thus, `(cons 2 3) = (2 . 3)`, and `(cdr '(2 . 3)) = 3`. The list `(2 3 4)` can then be written in pair notation as `(2 . (3 . (4 . ())))`. Discuss any advantages you can think of to this more general data structure.

- 3.16** Write a Scheme list representation for the binary search tree shown in Figure 3.11.



**Figure 3.11** Binary search tree for Exercise 3.16

- 3.17** Write an `insert` procedure in Scheme for the binary search tree data structure described in Section 3.3.2.
- 3.18** Try to write a tail-recursive version of the `append` function in Scheme. What problems do you encounter? Can you think of a list representation that would make this easier?
- 3.19** (a) Give an algebraic specification for the ADT `List` with the following operations, and with properties as in Scheme: `car`, `cdr`, `cons`, `null?`, `makemul`.  
 (b) Write a C++ or Java class, or an Ada package to implement the ADT of (a).  
 (c) Use your implementation of the `List` ADT in (b) to write a “tiny Scheme” interpreter, that is, an interpreter that has only the ADT list functions as available operations and only has numbers as atoms.
- 3.20** In Section 3.2.6 we defined the `make-double` higher-order function in Scheme. What functions are returned by `(make-double -)` and `(make-double /)`?
- 3.21** Scheme does not have a built-in `filter` function. Like `map`, `filter` is a higher-order function that expects a function and a list as arguments and returns a list of results, but the function argument to `filter` is a predicate or Boolean function of one argument. `filter` uses the predicate to test each element in the list and returns a list of just those elements that pass the test. Define a `filter` function in Scheme.
- 3.22** (a) Give an example to show that Scheme does not curry its functions.  
 (b) Write a Scheme higher-order function that takes a function of two parameters as its parameter and returns a curried version of the function.
- 3.23** List the ML types for all the functions in Exercise 3.8.
- 3.24** Write an ML function that determines the length of any list. Show how an ML system can determine the type of the length function using pattern matching.
- 3.25** List the Haskell types for all the functions in Exercise 3.8.
- 3.26** Write Haskell list comprehensions for the following lists: (a) all integers that are squares, (b) all Pythagorean triples (see Exercise 3.8), and (c) all perfect numbers (a perfect number is the sum of all of its proper factors).
- 3.27** (a) Write functions to test whether ML and Scheme use short-circuit evaluation for Boolean expressions.  
 (b) Why do we not need to test Haskell to answer this question for that language?
- 3.28** When a function is defined using pattern matching in ML, the text mentions that an ML system may complain if cases are left out of the definition. Such missing cases imply the function is partial. Discuss the advantages and disadvantages of requiring all functions to be total in a programming language.
- 3.29** The `fact` function defined in this chapter for ML and Scheme is in fact partial, yet an ML translator will not discover this. Why?

- 3.30** ML and Haskell do not have general list structures like Scheme, but require the elements in a list to have all the same data type. Why is this? What data structure in an imperative language do such lists imitate?
- 3.31** Write a Scheme program to show that the Scheme procedures `force` and `delay` actually use pass by need memoization.
- 3.32** (a) Write a sieve of Eratosthenes in Scheme or ML using generators and filters, similar to the Haskell version in Section 3.5.  
 (b) Rewrite the Scheme version to use `force` and `delay`.
- 3.33** Rewrite the Scheme `intlist` function in Section 3.4 so that it takes only a lower bound as a parameter and produces a stream of integers from that point on: `(intlist 5) = (5 6 7 8 ...)`.
- 3.34** Haskell list comprehensions are actually compact expressions for generator-filter programs as noted in the text. For example, the list comprehension
- ```
evens = [ n | n <- [2..], mod n 2 == 0]
```
- is equivalent to a generator procedure that produces the stream of integers beginning with 2 (represented by `[2..]`) and sends its output to the selector procedure of the predicate `mod n 2 == 0` that passes on the list whose elements satisfy the predicate. Write a Scheme procedure that uses `force` and `delay` to produce the stream of even integers in a similar generator-filter style.
- 3.35** Rewrite the `flatten` Scheme procedure of Section 3.4 to use `force` and `delay`.
- 3.36** (From Abelson and Sussman [1996]) Define a delayed version of the `map` function from Section 3.2.6 as follows:

```
(define (delayed-map f L)
  (if (null? L) '()
      (delay (cons (f (car (force L)))
                    (delayed-map f (cdr (force L)))))))
```

Now define a `show` procedure that prints a value and returns the same value:

```
define (show x) (display x) (newline) x)
```

Finally, define a delayed list as follows:

```
(define L
  (delayed-map show (delay (intlist 1 100))))
```

where `intlist` is the delayed version from Section 3.4. What will the Scheme interpreter print when given the following expressions to evaluate in the order given (`take` is also the delayed version in Section 3.4):

```
> (take 5 L)
... some output here
> (take 7 L)
... some more output here
```

(Hint: The answer depends on whether Scheme uses pass by need or not; see Exercise 3.31.)

- 3.37** Write lambda calculus expressions for the higher-order functions `twice` and `inc_n`. (See Exercise 3.8.)
- 3.38** Assume `square = (λx. * x x)` in lambda calculus. Show the steps in an applicative order and normal order reduction of the expression `(twice (twice square))`. (See the previous exercise.)
- 3.39** Give applicative and normal order reductions for the following lambda expressions. State which steps use which conversions and which variables are bound and which are free.
- (a) $(\lambda x. ((\lambda y. (* 2 y)) (+ x y))) y$
- (b) $(\lambda x. \lambda y. (x y)) (\lambda z. (z y))$
- 3.40** It is a surprising fact in lambda calculus that lists can be expressed as lambda abstractions. Indeed, the list constructor `cons` can be written as $(\lambda x. \lambda y. \lambda f. f x y)$. With this definition one can write `car` as $(\lambda z. z (\lambda x. \lambda y. x))$ and `cdr` as $(\lambda z. z (\lambda x. \lambda y. y))$. Show that using these definitions the usual formulas $(\text{car } (\text{cons } a b)) = a$ and $(\text{cdr } (\text{cons } a b)) = b$ are true.
- 3.41** (From Abelson and Sussman [1996]) It is also possible to express the integers as lambda abstractions:

$\text{zero} = \lambda f. \lambda x. x$

$\text{one} = \lambda f. \lambda x. (f x)$

$\text{two} = \lambda f. \lambda x. (f (f x))$

...

These are called Church numbers, after Alonzo Church.

- (a) Given the following definition of the successor function:

$$\text{successor} = \lambda n. \lambda f. \lambda x. (f ((n f) x))$$

show that $(\text{successor zero}) 5$ one and $(\text{successor one}) 5$ two.

- (b) Generalize (a) to any Church number.
- (c) Define addition and multiplication for Church numbers.
- (d) Write out an implementation of your lambda expressions in (c) as procedures in a functional language, and write an output procedure that shows they are correct.
- 3.42** Use the lambda expression $H = (\lambda F. \lambda n. (\text{if } (= n 0) 1 (* n (F (- n 1)))))$ and the property that the fact function is a fixed point of H to show that $\text{fact } 1 = 1$.
- 3.43** We noted that the fixed-point combinator Y can itself be written as the lambda abstraction $(\lambda h. (\lambda x. h (x x)) (\lambda x. h (x x)))$. Show that this expression for Y does indeed give it the property that $Y H = H (Y H)$, for any H .
- 3.44** Consider writing the fixed-point combinator in Scheme, ML, or Haskell by using the fixed-point property $y h = h (y h)$ as the definition.
- (a) What is the type of y in ML or Haskell?
- (b) Using the definition of h for the factorial function:

$$h \ g \ n = \text{if } n = 0 \text{ then } 1 \text{ else } n * g(n - 1)$$

does $y h$ actually produce the correct factorial function in Scheme, ML, or Haskell? Explain.

Notes and References

There are many books on functional programming. We list only a few here that are directly related to the languages and issues discussed in the chapter. For an overview of functional programming languages, including historical perspectives, from a somewhat different viewpoint from this chapter, see Hudak [1989] and MacLennan [1990]. Reade [1989] is a comprehensive text on functional programming, with many examples from ML. Abelson and Sussman [1996] is also a good reference for much of this chapter, as well as for the Scheme language (Section 3.2). Friedman et al. [1996] is an advanced reference for Scheme. The (current) definition of Scheme is published in Sperber, Dybvig, Flatt, and van Stratten [2009]. The Scheme R6RS report can be found online at <http://www.r6rs.org/final/html/r6rs/r6rs.html>.

Functional programming with the ML language (Section 3.3) is treated in Paulson [1996] and Ullman [1998]. Milner et al. [1997] gives the definition of Standard ML97, and Milner and Tofte [1991] provides a description of some of the theoretical issues surrounding the design of ML; see also Mitchell and Harper [1988].

The Haskell language (Section 3.5) is presented in Hudak [2000], Thompson [1999], and Bird et al. [1998]. The influential predecessor language Miranda is described in Turner [1986], with underlying ideas presented in Turner [1982].

General techniques for functional programming are studied in Okasaki [1999] and Lapalme and Rabhi [1999]. Implementation techniques for functional languages are studied in Peyton Jones [1987] and Appel [1992].

Delayed evaluation is studied in Henderson [1980]; parts of Section 3.4 are adapted from that reference.

Interest in functional programming increased dramatically following the ACM Turing Award lecture of Backus [1978], in which he describes a general framework for functional programming, called FP. A significant modern version of Lisp not discussed in this chapter is Common Lisp, which is described in Steele [1982], Graham [1996], and Seibel [2005] and defined in Steele [1984]. The ANSI Standard for Common Lisp is ANSI X3.226 [1994].

The lambda calculus (Section 3.6) began with Church [1941] and is studied in Curry and Feys [1958], Barendregt [1982], and Hankin [1995]. Overviews of lambda calculus are presented in Peyton Jones [1987], where the use of normal forms in compiling Miranda is discussed; parts of the presentation in Section 3.6 are patterned after that text. Gunter [1992] and Sethi [1995] also survey the lambda calculus and study a statically typed version called the typed lambda calculus. Paulson [1996] describes an interpreter for lambda calculus. A history of the lambda calculus and the Church-Rosser theorem is given in Rosser [1982]. Curry is apparently originally due not to Curry but to Schönfinkel [1924].

Traditionally, functional programming has been thought to be highly inefficient because of its reliance on function call and dynamic memory management. Advances in algorithms, such as generational garbage collection, and advances in translation techniques make this less true. Some references dealing with these issues are Steele [1977] and Gabriel [1985]. Using recursive calls to do functional programming in an imperative language is also less expensive than one might imagine, and the techniques in Section 3.1 are usable in practice. An example of this is discussed in Loudon [1987].

CHAPTER

Logic Programming

4.1	Logic and Logic Programs	105
4.2	Horn Clauses	109
4.3	Resolution and Unification	111
4.4	The Language Prolog	115
4.5	Problems with Logic Programming	126
4.6	Curry: A Functional Logic Language	131

Logic, the science of reasoning and proof, has existed since the time of the ancient Greek philosophers. Mathematical, or symbolic, logic, the theory of mathematical proofs, began with the work of George Boole and Augustus De Morgan in the middle of the 1800s. Since then, logic has become a major mathematical discipline and has played a significant role in the mathematics of the twentieth century, particularly with respect to the famous incompleteness theorems of Kurt Gödel. (See the Notes and References.)

Logic is closely associated with computers and programming languages in a number of ways. First, computer circuits are designed with the help of Boolean algebra, and Boolean expressions and data are almost universally used in programming languages to control the actions of a program. Second, logical statements are used to describe **axiomatic semantics**, or the semantics of programming language constructs. Logical statements can also be used as formal specifications for the required behavior of programs, and together with the axiomatic semantics of the language, they can be used to prove the correctness of a program in a purely mathematical way.

In the opposite direction, computers are used as tools to implement the principles of mathematical logic, with programmers creating programs that can construct proofs of mathematical theorems using the principles of logic. These programs, known as **automatic deduction systems** or **automatic theorem provers**, turn proofs into computation. Experience with such programs in the 1960s and 1970s led to the major realization that the reverse is also true: Computation can be viewed as a kind of proof. Thus, logical statements, or at least a restricted form of them, can be viewed as a programming language and executed on a computer, given a sophisticated enough interpretive system. This work, primarily by Robinson, Colmerauer, and Kowalski (see the Notes and References), led to the programming language **Prolog**. The development of efficient Prolog interpreters in the late 1970s, particularly at the University of Edinburgh, Scotland, resulted in a tremendous increase in interest in logic programming systems.

Then, in 1981, Prolog achieved almost instant fame when the Japanese government announced that it would serve as the base language for the Fifth Generation Project, whose goal was the development of advanced computer systems incorporating reasoning techniques and human language understanding. The project ended somewhat inconclusively in 1992. Nevertheless, a significant amount of research and development of logic programming systems occurred as a result. Prolog remains today the most significant example of a logic programming language.

In the following sections, we will first give a brief introduction to mathematical logic and the way logic can be used as a programming language. Next, we turn our attention to a description of Prolog and the techniques of writing Prolog programs. We also give a brief description of the principles behind the operation of a Prolog system, and some of the weaknesses of these systems. Finally, we describe some of the additional work on equational and constraint logic systems.

4.1 Logic and Logic Programs

Before you can learn about logic programming, you need to know a little bit about mathematical logic. The kind of logic used in logic programming is the **first-order predicate calculus**, which is a way of formally expressing **logical statements**, that is, statements that are either true or false.

EXAMPLE 1

The following English statements are logical statements:

0 is a natural number.

2 is a natural number.

For all x , if x is a natural number, then so is the successor of x .

21 is a natural number.

A translation into predicate calculus is as follows:

`natural(0).`

`natural(2).`

For all x , `natural(x)  $\rightarrow$  natural(successor (x)).`

`natural(21).`

Among these logical statements, the first and third statements can be viewed as **axioms** for the natural numbers: statements that are assumed to be true and from which all true statements about natural numbers can be **proved**. Indeed, the second statement can be proved from these axioms, since `2 = successor(successor(0))` and `natural(0)  $\rightarrow$  natural (successor(0))  $\rightarrow$  natural (successor(successor (0)))`. The fourth statement, on the other hand, cannot be proved from the axioms and so can be assumed to be false.

First-order predicate calculus classifies the different parts of such statements as follows:

1. *Constants*. These are usually numbers or names. Sometimes they are called atoms, since they cannot be broken down into subparts. In Example 1, 0 is a constant.
2. *Predicates*. These are names for functions that are true or false, like Boolean functions in a program. Predicates can take a number of arguments. In Example 1, the predicate `natural` takes one argument.
3. *Functions*. First-order predicate calculus distinguishes between functions that are true or false—these are the predicates—and all other functions, like `successor` in Example 1, which represent non-Boolean values.
4. *Variables That Stand for as yet Unspecified Quantities*. In Example 1, x is a variable.
5. *Connectives*. These include the operations and, or, and not, just like the operations on Boolean data in programming languages. Additional connectives in predicate calculus are implication, signified by $\rightarrow$, and equivalence, indicated by $\leftrightarrow$. These are not really new operations: $a \rightarrow b$ means that b is true whenever a is true. This is equivalent to the statement b or not a . Also $a \leftrightarrow b$ means the same as $(a \rightarrow b)$ and $(b \rightarrow a)$.

6. *Quantifiers*. These are operations that introduce variables. In Example 1, `forall` is the quantifier for `x`; it is called the **universal quantifier**. The statement:

`forall x, natural(x) → natural(successor(x))`

means that for every `x` in the universe, if `x` is a natural number, then the successor of `x` is also a natural number. A universal quantifier is used to state that a relationship among predicates is true for all things in the universe named by the variable `x`.

There is also the **existential quantifier**, `exists`, as in the following statement:

`exists x, natural(x).`

This statement means that there exists an `x` such that `x` is a natural number. An existential quantifier is used to state that a predicate is true of at least one thing in the universe, indicated by the variable `x`.

A variable introduced by a quantifier is said to be **bound** by the quantifier. It is possible for variables also to be **free**, that is, not bound by any quantifier.

7. *Punctuation Symbols*. These include left and right parentheses, the comma, and the period. (Strictly speaking, the period isn't necessary, but we include it since most logic programming systems use it.) Parentheses are used to enclose arguments and also to group operations. Parentheses can be left out in accordance with common conventions about the precedence of connectives, which are usually assumed to be the same as in most programming languages. (Thus, the connectives in order of decreasing precedence are `not`, `and`, `or`, `→`, and `↔`.)

In predicate calculus, arguments to predicates and functions can only be **terms**—that is, combinations of variables, constants, and functions. Terms cannot contain predicates, quantifiers, or connectives. In Example 1, terms that appear include constants `0`, `21`, `2`, the variable `x`, and the term `successor(x)`, consisting of the function `successor` with argument `x`. Some examples of additional terms that can be written are `successor(0)` and `successor(successor(successor(x)))`.

EXAMPLE 2

The following are logical statements in English:

A horse is a mammal.

A human is a mammal.

Mammals have four legs and no arms, or two legs and two arms.

A horse has no arms.

A human has arms.

A human has no legs.

A possible translation of these statements into first-order predicate calculus is as follows:

```
mammal(horse).
mammal(human).
for all x, mammal(x) →
    legs(x, 4) and arms(x, 0) or legs(x, 2) and arms(x, 2).
arms(horse, 0).
not arms(human, 0).
legs(human, 0).
```

In this example, the constants are the integers 0, 2, and 4 and the names horse and human. The predicates are mammal, arms, and legs. The only variable is x , and there are no functions.

As in the previous example, we might consider the first five statements to be axioms—statements defining true relationships. Then, as we shall see shortly, $\text{arms}(\text{human}, 2)$ becomes provable from the axioms. It is also possible to prove that $\text{legs}(\text{human}, 2)$ is true, so the last statement is false, given the axioms.

Note that we have used precedence to leave out many parentheses in the preceding statements, so that, for example, when we write

```
legs(x, 4) and arms(x, 0) or legs(x, 2) and arms(x, 2)
```

we mean the following:

```
(legs(x,4) and arms(x,0)) or (legs(x,2) and arms(x,2)).
```

In addition to the seven classes of symbols described, first-order predicate calculus has **inference rules**, which are ways of deriving or proving new statements from a given set of statements.

EXAMPLE 3

A typical inference rule is the following:

From the statements $a \rightarrow b$ and $b \rightarrow c$, one can derive the statement $a \rightarrow c$, or written more formally,

$$\frac{(a \rightarrow b) \text{ and } (b \rightarrow c)}{a \rightarrow c}$$

Inference rules allow us to construct the set of all statements that can be derived, or proved, from a given set of statements: These are statements that are always true whenever the original statements are true. For example, the first five statements about mammals in Example 2 allow us to derive the following statements:

```
legs(horse, 4).
legs(human, 2).
arms (human, 2).
```

Stated in the language of logic, these three statements become **theorems** derived from the first five statements or axioms of Example 2. Notice that proving these statements from the given statements can be viewed as the computation of the number of arms and legs of a horse or a human. Thus, the

set of statements in Example 2 can be viewed as representing the potential computation of all logical consequences of these statements.

This is the essence of logic programming: A collection of statements is assumed to be axioms, and from them a desired fact is derived by the application of inference rules in some automated way. Thus, we can state the following definition:

A **logic programming language** is a notational system for writing logical statements together with specified algorithms for implementing inference rules.

The set of logical statements that are taken to be axioms can be viewed as the **logic program**, and the statement or statements that are to be derived can be viewed as the input that initiates the computation. Such inputs are also provided by the programmer and are called **queries** or **goals**. For example, given the set of axioms of Example 2, if we wanted to know how many legs a human has, we would provide the following query:

Does there exist a y such that y is the number of legs of a human?

or, in predicate calculus,

there exists y , $\text{legs}(\text{human}, y)$?

and the system would respond with something like:

yes: y 5 2

For this reason, logic programming systems are sometimes referred to as **deductive databases**, databases consisting of a set of statements and a deduction system that can respond to queries. Notice that these are different from ordinary databases, since they contain not only facts such as $\text{mammal}(\text{human})$ or $\text{natural}(0)$ but also more complex statements such as $\text{natural}(x) \rightarrow \text{natural}(\text{successor}(x))$, and the system can answer not only queries about facts but also queries involving such implications.

In a pure logic programming system, nothing is said about how a particular statement might be derived from a given set of statements. The specific path or sequence of steps that an automatic deduction system chooses to derive a statement is the **control problem** for a logic programming system. The original statements represent the logic of the computation, while the deductive system provides the control by which a new statement is derived. This property of logic programming systems led Kowalski to state the logic programming paradigm as the pseudoequation:

$$\text{algorithm} = \text{logic} + \text{control}$$

as a contrast to Niklaus Wirth's expression of imperative programming as:

$$\text{algorithms} = \text{data structures} + \text{programs}$$

(See the Notes and References.) Kowalski's principle points out a further feature of logic programming: Since logic programs do not express the control, operations (in theory at least) can be carried out in any order or simultaneously. Thus, logic programming languages are natural candidates for parallelism.

Unfortunately, automated deduction systems have difficulty handling all of first-order predicate calculus. First, there are too many ways of expressing the same statements, and second, there are too many inference rules. As a result, most logic programming systems restrict themselves to a particular subset of predicate calculus, called Horn clauses, which we will study briefly next.

4.2 Horn Clauses

A **Horn clause** (named after its inventor Alfred Horn) is a statement of the form:

$$a_1 \text{ and } a_2 \text{ and } a_3 \dots \text{ and } a_n \rightarrow b$$

where the a_i are only allowed to be simple statements involving no connectives. Thus, there are no *or* connectives and no quantifiers in Horn clauses. The preceding Horn clause says that a_1 through a_n imply b , or that b is true if all the a_i are true. b is called the **head** of the clause, and the $a_1 \dots, a_n$ is the **body** of the clause. In the Horn clause, the number of a_i 's may be 0, in which case the Horn clause has the form:

$$\rightarrow b$$

Such a clause means that b is always true. In other words, b is an axiom and is usually written without the connective $\rightarrow$. Such clauses are sometimes also called **facts**.

Horn clauses can be used to express most, but not all, logical statements. Indeed, one algorithm that is beyond the scope of this book can perform a reasonable translation from predicate calculus statements to Horn clauses. (See the Notes and References.) The basic idea is to remove *or* connectives by writing separate clauses, and to treat the lack of quantifiers by assuming that variables appearing in the head of a clause are universally quantified, while variables appearing in the body of a clause (but not in the head) are existentially quantified.

EXAMPLE 4

The following statements from Example 1 are written in first-order predicate calculus:

```
natural(0).
for all x, natural(x) → natural (successor (x)).
```

These can be very simply translated into Horn clauses by dropping the quantifier:

```
natural(0).
natural(x) → natural (successor(x)).
```

EXAMPLE 5

Consider the logical description for the Euclidian algorithm to compute the greatest common divisor of two positive integers u and v :

The gcd of u and 0 is u .

The gcd of u and v , if v is not 0, is the same as the gcd of v and the remainder of dividing v into u .

Translating this into first-order predicate calculus gives:

```
for all u, gcd(u, 0, u).
for all u, for all v, for all w,
not zero(v) and gcd(v, u mod v, w) → gcd(u, v, w).
```

(Remember that $\text{gcd}(u, v, w)$ is a predicate expressing that w is the gcd of u and v .)

To translate these statements into Horn clauses, we need again only drop the quantifiers:

```
gcd(u, 0, u).
not zero(v) and gcd(v, u mod v, w) → gcd(u, v, w).
```

EXAMPLE 6

The foregoing examples contain only universally quantified variables. To see how an existentially quantified variable in the body may also be handled, consider the following statement:

x is a grandparent of y if x is the parent of
someone who is the parent of y .

Translating this into predicate calculus, we get

for all x , for all y , (there exists z , $\text{parent}(x, z)$ and $\text{parent}(z, y)$)
 $\rightarrow \text{grandparent}(x, y)$.

As a Horn clause this is expressed simply as:

$\text{parent}(x, z)$ and $\text{parent}(z, y) \rightarrow \text{grandparent}(x, y)$. ■

EXAMPLE 7

To see how connectives are handled, consider the following statement:

For all x , if x is a mammal then x has two or four legs.

Translating in predicate calculus, we get:

for all x , $\text{mammal}(x) \rightarrow \text{legs}(x, 2) \text{ or } \text{legs}(x, 4)$.

This may be approximated by the following Horn clauses:

$\text{mammal}(x)$ and not $\text{legs}(x, 2) \rightarrow \text{legs}(x, 4)$.
 $\text{mammal}(x)$ and not $\text{legs}(x, 4) \rightarrow \text{legs}(x, 2)$. ■

In general, the more connectives that appear to the right of a $\rightarrow$ connective in a statement, the harder it is to translate into a set of Horn clauses; see Exercise 4.8.

Horn clauses are of particular interest to automatic deduction systems such as logic programming systems, because they can be given a **procedural interpretation**. If we write a Horn clause in reverse order

$$b \leftarrow a_1 \text{ and } a_2 \text{ and } a_3 \dots \text{ and } a_n$$

we can view this as a definition of procedure b : the body of b is given by the body of the clause, namely the operations indicated by the a_i 's. This is very similar to the way context-free grammar rules are interpreted as procedure definitions in recursive descent parsing, as we shall see in Chapter 6. There is more than a passing similarity here, since logic programs can be used to directly construct parsers. (See the Notes and References.) In fact, the parsing of natural language was one of the motivations for the original development of Prolog. The major difference between parsing and logic programming is that in pure logic programming, the order in which the a_i 's are called is not specified.

Nevertheless, most logic programming systems are deterministic in that they perform the calls in a certain prespecified order, usually left to right, which is exactly the order indicated by context-free grammar rules. The particular kind of grammar rules used in Prolog programs are called **definite clause grammars**.

Horn clauses can also be viewed as **specifications** of procedures rather than strictly as implementations. For example, we could view the following Horn clause as a specification of a sort procedure:

$\text{sort}(x, y) \leftarrow \text{permutation}(x, y) \text{ and } \text{sorted}(y)$.

This says that (assuming that x and y are lists of things) a sort procedure transforms list x into a sorted list y such that y is a permutation of x . To complete the specification, we must, of course, supply

specifications for what it means for a list to be sorted and for a list to be a permutation of another list. The important point is that we may think of the Horn clause as not necessarily supplying the algorithm by which y is found, given an x , but only the properties such a y must have.

With the foregoing procedural interpretation in mind, most logic programming systems not only write Horn clauses backward but also drop the *and* connectives between the a_i , separating them with commas instead. Thus, the greatest common divisor clauses in Example 5 would appear as follows:

```
gcd(u, 0, u).
gcd(u, v, w) ← not zero(v), gcd(v, u mod v, w).
```

This is beginning to look suspiciously like a more standard programming language expression for the gcd, such as

```
gcd(u, v) = if v = 0 then u else gcd(v, u mod v).
```

From now on we will write Horn clauses in this form.

Now let's consider the issue of variable scope in a procedural interpretation of Horn clauses. As with procedure definitions in a block-structured language, the assumption is that all variables are local to each call of the procedure. Indeed, variables used in the head can be viewed as parameters, while variables used only in the body can be viewed as local, temporary variables. This is only an approximate description, as the algorithms used to implement the execution of Horn clauses treat variables in a more general way, as you will see in Section 4.3.

We haven't yet seen how queries or goal statements can be expressed as Horn clauses. In fact, a query is exactly the opposite of a fact—a Horn clause with no head:

```
mammal(human) ←. — a fact
mammal(human). — a query or goal
```

A Horn clause without a head could also include a sequence of queries separated by commas:

```
← mammal(x), legs(x, y).
```

Why queries correspond to Horn clauses without heads should become clear when you understand the inference rule that logic programming systems apply to derive new statements from a set of given statements, namely, the resolution rule or principle, studied next.

4.3 Resolution and Unification

Resolution is an inference rule for Horn clauses that is especially efficient. Resolution says that if we have two Horn clauses, and we can match the head of the first Horn clause with one of the statements in the body of the second clause, then the first clause can be used to replace its head in the second clause by its body. In symbols, if we have Horn clauses:

$$\begin{aligned} a &\leftarrow a_1, \dots, a_n. \\ b &\leftarrow b_1, \dots, b_m. \end{aligned}$$

and b_i matches a , then we can infer the clause:

$$b \leftarrow b_1, \dots, b_{i-1}, a_1, \dots, a_n, b_{i+1}, \dots, b_m.$$

The simplest example of this occurs when the body of the Horn clauses contains only single statements, such as in:

$$b \leftarrow a.$$

and:

$$c \leftarrow b.$$

In this case, resolution says that we may infer the following clause:

$$c \leftarrow a.$$

This is precisely the inference rule that we gave in Example 3 in Section 4.1.

Another way of looking at resolution is to combine left-hand and right-hand sides of both Horn clauses and then cancel those statements that match on both sides. Thus, for the simplest example,

$$b \leftarrow a.$$

and:

$$c \leftarrow b.$$

give:

$$b, c \leftarrow a, b.$$

and canceling the b ,

$$\cancel{b}, c \leftarrow a, \cancel{b}$$

gives:

$$c \leftarrow a.$$

Now you can see how a logic programming system can treat a goal or list of goals as a Horn clause without a head. The system attempts to apply resolution by matching one of the goals in the body of the headless clause with the head of a known clause. It then replaces the matched goal with the body of that clause, creating a new list of goals, which it continues to modify in the same way. The new goals are called **subgoals**. In symbols, if we have the goal:

$$\leftarrow a.$$

and the clause $a \leftarrow a_1, \dots, a_n$, then resolution replaces the original goal a with the subgoals:

$$\leftarrow a_1, \dots, a_n.$$

If the system succeeds eventually in eliminating all goals—thus deriving the empty Horn clause—then the original statement has been proved.

EXAMPLE 8

The simplest case is when the goal is already a known fact, such as:

`mammal(human).`

and one asks whether a human is a mammal:

`← mammal(human).`

Using resolution, the system combines the two Horn clauses into:

`mammal(human) ← mammal(human).`

and then cancels both sides to obtain:

`←.`

Thus, the system has found that indeed a human is a mammal and would respond to the query with “Yes.”

EXAMPLE 9

A slightly more complicated example is the following. Given the rules and facts:

`legs(x, 2) ← mammal(x), arms(x, 2).`

`legs(x, 4) ← mammal(x), arms(x, 0).`

`mammal(horse).`

`arms(horse, 0).`

if we supply the query:

`← legs(horse, 4).`

then applying resolution using the second rule, we get:

`legs(x, 4) ← mammal(x), arms(x, 0), legs(horse, 4).`

Now, to cancel the statements involving the predicate `legs` from each side, we have to match the variable `x` to `horse`, so we replace `x` by `horse` everywhere in the statement:

`legs(horse, 4) ← mammal(horse), arms(horse, 0), legs(horse, 4).`

and cancel to get the subgoals:

`← mammal(horse), arms(horse, 0).`

Now we apply resolution twice more, using the facts `mammal(horse)` and `arms(horse, 0)` and cancel:

`mammal(horse) ← mammal(horse), arms(horse, 0).`

`arms(horse, 0).`

`arms(horse, 0) ← arms(horse, 0).`

Since we have arrived at the empty statement, our original query is true.

Example 9 demonstrates an additional requirement when we apply resolution to derive goals. To match statements that contain variables, we must set the variables equal to terms so that the statements become identical and can be canceled from both sides. This process of pattern matching to make statements identical is called **unification**, and variables that are set equal to patterns are said to be **instantiated**. Thus, to implement resolution effectively, we must also provide an algorithm for unification. A very general unification algorithm does exist and was mentioned in Chapter 3 in relation to the Hindley-Milner type inference. Most logic programming systems, however, use a slightly weaker algorithm, which we discuss in Section 4.4.

EXAMPLE 10

To show in more detail the kind of unification that takes place in a logic programming language, consider the greatest common divisor problem (Euclid's algorithm, Example 5):

$$\text{gcd}(u, 0, u).$$

$$\text{gcd}(u, v, w) \leftarrow \text{not zero}(v), \text{gcd}(v, u \bmod v, w).$$

Now given the goal:

$$\leftarrow \text{gcd}(15, 10, x).$$

resolution fails using the first clause (10 does not match 0), so using the second clause and unifying $\text{gcd}(u, v, w)$ with $\text{gcd}(15, 10, x)$ gives:

$$\text{gcd}(15, 10, x) \leftarrow \text{not zero}(10), \text{gcd}(10, 15 \bmod 10, x), \text{gcd}(15, 10, x).$$

Assuming that the system knows that $\text{zero}(10)$ is false, so that $\text{not zero}(10)$ is true, and simplifying $15 \bmod 10$ to 5, we cancel $\text{gcd}(15, 10, x)$ from both sides and obtain the subgoal:

$$\leftarrow \text{gcd}(10, 5, x).$$

Note that this is just another goal to be resolved like the original goal. This we do by unification as before, obtaining

$$\text{gcd}(10, 5, x) \leftarrow \text{not zero}(5), \text{gcd}(5, 10 \bmod 5, x), \text{gcd}(10, 5, x).$$

and so get the further subgoal:

$$\leftarrow \text{gcd}(5, 0, x).$$

Now this matches the first rule,

$$\text{gcd}(u, 0, u).$$

so instantiating x to 5 results in the empty statement, and the system will reply with something like:

Yes: $x = 5$

Resolution and unification have performed the computation of the greatest common divisor of 10 and 15, using Euclid's algorithm!

Now let's consider another problem that must be solved before a working resolution implementation can be achieved. To achieve efficient execution, a logic programming system must apply a fixed algorithm that specifies: (1) the order in which the system attempts to resolve a list of goals, and (2) the order in which clauses are used to resolve goals. As an example of (1), consider the goal:

```
← legs(horse, 4).
```

In Example 9, this led to the two subgoals:

```
← mammal(horse), arms(horse, 0).
```

A logic programming system must now decide whether to try to resolve `mammal(horse)` first or `arms(horse,0)` first. In this example, both choices lead to the same answer. However, as you will see in the examples that follow, the order can have a significant effect on the answers found.

The order in which clauses are used can also have a major effect on the result of applying resolution. For example, given the Horn clauses:

```
ancestor(x, y) ← parent(x, z), ancestor(z, y).
ancestor(x, x).
parent(amy, bob).
```

if we provide the query:

```
← ancestor(x, bob).
```

there are two possible answers: `x = bob` and `x = amy`. If the assertion `ancestor(x, x)` is used, the solution `x = bob` will be found. If the clause `ancestor(x, y) ← parent(x, z), ancestor(z, y)` is used, the solution `x = amy` will be found. Which one is found first, or even if any is found, can depend on the order in which the clauses are used, as well as the order in which goals are resolved.

Logic programming systems using Horn clauses and resolution with prespecified orders for (1) and (2), therefore, violate the basic principle that such systems set out to achieve: that a programmer need worry only about the logic itself, while the control (the methods used to produce an answer) can be ignored. Instead, a programmer must always be aware of the way the system produces answers. It is possible to design systems that will always produce all the answers implied by the logic, but they are (so far) too inefficient to be used as programming systems.

4.4 The Language Prolog

Prolog is the most widely used logic programming language. It uses Horn clauses and implements resolution via a strictly linear depth-first strategy and a unification algorithm described in more detail shortly. Although an ISO standard for Prolog now exists, for many years there was no standard, and implementations contained many variations in the details of their syntax, semantics, built-in functions, and libraries. This is still the case, but the most widely used implementation (the so-called **Edinburgh Prolog** developed in the late 1970s and early 1980s at the University of Edinburgh) was somewhat of a de facto standard and indeed was used as the basis for the ISO standard. We use its notation for the clauses and goals in our examples below. We discuss the essential features of Prolog in the sections that follow.

4.4.1 Notation and Data Structures

Prolog uses notation almost identical to that developed earlier for Horn clauses, except that the implication arrow, $\leftarrow$, is replaced by a colon followed by a dash (or minus sign) ($:-$). Thus, the sample ancestor program from the previous section would be written in Prolog syntax as follows:

```
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).
ancestor(X, X).
parent(amy, bob).
```

and Example 4 would be written as follows:

```
natural(0).
natural(successor(X)) :- natural(X).
```

Note that the variables X and Y are written in uppercase. Prolog distinguishes variables from constants and names of predicates and functions by using uppercase for variables and lowercase for constants and names. It is also possible in most Prolog systems to denote a variable by writing an underscore before the name, as in `ancestor(_X, _X)`.

In addition to the comma connective that stands for and, Prolog uses the semicolon for or. However, the semicolon is rarely used in programming, since it is not a standard part of Horn clause logic.

Basic data structures are terms like `parent(X, Z)` or `successor(successor(0))`. Prolog also includes the list as a basic data structure. A Prolog list is written using square brackets (like ML and Haskell, with the list consisting of items x , y , and z written as `[x, y, z]`). Lists may contain terms or variables.

It is also possible to specify the head and tail of a list using a vertical bar. For example, `[H|T]` means that H is the first item in the list and T is the tail of the list. As in ML and Haskell, this type of notation can be used to extract the components of a list via pattern matching. Thus, if `[H|T] = [1, 2, 3]`, then $H = 1$ and $T = [2, 3]$. It is also possible to write as many terms as one wishes before the bar. For example, `[X,Y|Z] = [1, 2, 3]` gives $X = 1$, $Y = 2$, and $Z = [3]$. The empty list is denoted by `[]` and does not have a first element.

Prolog has a number of standard predicates that are always built in, such as `not`, `=`, and the I/O operations `read`, `write`, and `nl` (for newline). One anomaly in Prolog is that the less than or equal to operator is usually written `=<` instead of `<=` (perhaps because the latter is too easily confused with implication).

4.4.2 Execution in Prolog

Prolog compilers exist, but most systems are run as interpreters. A Prolog program consists of a set of Horn clauses in Prolog syntax, which is usually entered from a file and stored in a dynamically maintained database of clauses. Once a set of clauses has been entered into the database, goals can be entered either from a file or from the keyboard to begin execution. Thus, once a Prolog system has begun to execute, it will provide the user with a prompt for a query, such as

```
?- _
```

Note that, while Prolog syntax is now standardized for clauses and queries, the responses from a Prolog interpreter (including the prompt as shown above) are *not* standardized; we give these responses in a plausible notation, but different ISO-compliant interpreters may behave slightly differently.

EXAMPLE 11

If the clauses

```
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).
ancestor(X, X).
parent(amy, bob).
```

have been entered into the database, then the following queries would cause the indicated response:

```
?- ancestor(amy, bob).
yes.
```

```
?- ancestor(bob, amy).
no.
```

```
?- ancestor(X, bob).
X = amy ->_
```

In the last query there are two answers. Most Prolog systems will find one answer and then wait for a user prompt before printing more answers. If the user supplies a semicolon at the underscore (meaning or), then Prolog continues to find more answers:

```
?- ancestor(X, bob).
X = amy ->;
X = bob
?- _
```

A carriage return usually cancels the continued search. ■

4.4.3 Arithmetic

Prolog has built-in arithmetic operations and an arithmetic evaluator. Arithmetic terms can be written either in the usual infix notation or as terms in prefix notation: $3 + 4$ and $+(3, 4)$ mean the same thing. However, Prolog cannot tell when to consider an arithmetic term as a term itself (that is, strictly as data), or when to evaluate it. Thus,

```
?- write(3 + 5).
3 + 5
```

To force the evaluation of an arithmetic term, a new operation is required: the built-in predicate `is`. Thus, to get Prolog to evaluate $3 + 5$, we need to write:

```
?- X is 3 + 5, write(X).
X = 8
```

A further consequence of this is that two arithmetic terms may not be equal as terms, even though they have the same value¹:

```
?- 3 + 4 = 4 + 3.
no
```

To get equality of values, we must force evaluation using `is`, for example, by writing the predicate:

```
valequal(Term1, Term2) :-
    X is Term1, Y is Term2, X = Y.
```

We would then get:

```
?- valequal(3 + 4, 4 + 3).
yes
```

Now you can see how to write Euclid's algorithm for the greatest common divisor from Example 10 in Prolog. We wrote this algorithm in generic Horn clauses as:

```
gcd(u, 0, u).
gcd(u, v, w) ← not zero(v), gcd(v, u mod v, w).
```

In Prolog this translates to:

```
gcd(U, 0, U).
gcd(U, V, W) :-
    not(V = 0) , R is U mod V, gcd(V, R, W).
```

The middle statement `R is U mod V` is required to force evaluation of the `mod` operation.

4.4.4 Unification

Unification is the process by which variables are instantiated, or allocated memory and assigned values, so that patterns match during resolution. It is also the process of making two terms the same in some sense. The basic expression whose semantics is determined by unification is equality: In Prolog the goal `s = t` attempts to unify the terms `s` and `t`. It succeeds if unification succeeds and fails otherwise. Thus, we can study unification in Prolog by experimenting with the effect of equality:

```
?- me = me.
yes

?- me = you.
no

?- me = X.
X = me

?- f(a, X) = f(Y, b).
X = b
Y = a
```

(continues)

¹We write a space between the 3 and the period in the following goal, since 3. could be interpreted as the floating-point number 3.0.

(continued)

```
?- f(X) = g(X).
no

?- f(X) = f(a, b).
no

?- f(a, g(X)) = f(Y, b).
no

?- f(a, g(X)) = f(Y, g(b)).
X = b
Y = a
```

From these experiments, we can formulate the following unification algorithm for Prolog:

1. A constant unifies only with itself: $me = me$ succeeds but $me = you$ fails.
2. A variable that is uninstantiated unifies with anything and becomes instantiated to that thing.
3. A structured term (i.e., a function applied to arguments) unifies with another term only if it has the same function name and the same number of arguments, and the arguments can be unified recursively. Thus, $f(a, X)$ unifies with $f(Y, b)$ by instantiating X to b and Y to a .

A variation on case 2 is when two uninstantiated variables are unified:

```
?- X = Y.
X = _23
Y = _23
```

The number printed on the right-hand side—in this case, 23—will differ from system to system and indicates an internal memory location set aside for that variable. Thus, unification causes uninstantiated variables to share memory—that is, to become aliases of each other.

We can use unification in Prolog to get very short expressions for many operations. As examples, let us develop Prolog programs for the list operations **append** and **reverse**. We note again that because Prolog allows a list to be represented by a term such as $[X|Y]$, where X and Y are variables, there is no need for a built-in function to return the head of a list or the tail of a list: just setting $[X|Y] = [1, 2, 3]$ with uninstantiated variables X and Y returns the head as X and the tail as Y by unification. Similarly, there is no need for a list constructor like the `cons` operation of LISP. If we want to add 0, say, to the list $[1, 2, 3]$, we simply write $[0 | [1, 2, 3]]$; for example,

```
?- X = [0|[1, 2, 3]].
X = [0, 1, 2, 3]
```

However, we could, if we wanted, write the following clause:

```
cons(X, Y, L) :- L = [X|Y].
```

and then use it to compute heads, tails, and constructed lists, depending on how we instantiate the variables on the left-hand side:

```
?- cons(0, [1, 2, 3], A).
A = [0, 1, 2, 3]

?- cons(X, Y, [1, 2, 3]).
X = 1
Y = [2, 3]
```

Thus, we can use variables in a term as either input or output parameters, and Prolog clauses can be “run” backward as well as forward—something the procedural interpretation of Horn clauses did not tell us!

Indeed, unification can be used to shorten clauses such as `cons` further. Since the `=` operator is there only to force unification, we can let resolution cause the unification automatically by writing the patterns to be unified directly in the parameters of the head. Thus, the following definition of `cons` has precisely the same meaning as the previous definition:

```
cons(X, Y, [X|Y]).
```

The appearance of the pattern `[X|Y]` in place of the third variable automatically unifies it with a variable used in that place in a goal. This process could be referred to as **pattern-directed invocation**. The functional languages ML and Haskell studied in Chapter 3 have a similar mechanism, and both the runtime systems and the type checkers of these two languages use forms of unification similar to Prolog’s.

Now let’s write an `append` procedure:

```
append(X, Y, Z) :- X = [], Y = Z.
append(X, Y, Z) :-
    X = [A|B], Z = [A|W], append(B, Y, W).
```

The first clause states that appending any list to the empty list just gives that list. The second clause states that appending a list whose head is `A` and tail is `B` to a list `Y` gives a list whose head is also `A` and whose tail is `B` with `Y` appended.

Rewriting this using pattern-directed invocation, we get the following extremely concise form:

```
append([], Y, Y).
append([A|B], Y, [A|W]) :- append(B, Y, W).
```

This `append` can also be run backward and can even find all the ways to append two lists to get a specified list:

```
?- append(X, Y, [1, 2]).
X = []
Y = [1, 2] ->;

X = [1]
Y = [2] ->;

X = [1, 2].
Y = []
```

Prolog does this by first using the first clause and matching X to the empty list $[]$ and Y to the final list. It then continues the search for solutions by using the second clause, matching X to $[A|B]$ and setting up the subgoal $\text{append}(B, Y, W)$ with $W = [2]$. This begins the search over again with B in place of X , so B is first matched with $[]$ and Y with $[2]$, giving $X = [1|[]] = [1]$. B is then matched with a new $[A|B]$ by the second clause, and so on.

Finally, in this section, we give a Prolog definition for the reverse of a list:

```
reverse([], []).
reverse([H|T], L) :- reverse(T, L1),
                    append(L1, [H], L).
```

Figure 4.1 collects the three principal Prolog examples of this section, for ease of reference.

```
gcd(U, 0, U).
gcd(U, V, W) :- not(V = 0), R is U mod V, gcd(V, R, W).

append([], Y, Y).
append([A|B], Y, [A|W]) :- append(B, Y, W).

reverse([], []).
reverse([H|T], L) :- reverse(T, L1),
                    append(L1, [H], L).
```

Figure 4.1 Prolog clauses for gcd, append, and reverse

4.4.5 Prolog's Search Strategy

Prolog applies resolution in a strictly linear fashion, replacing goals left to right and considering clauses in the database in top-to-bottom order. Subgoals are also considered immediately once they are set up. Thus, this search strategy results in a depth-first search on a tree of possible choices.

For example, consider the following clauses:

- (1) $\text{ancestor}(X, Y) \text{ :- parent}(X, Z), \text{ancestor}(Z, Y).$
- (2) $\text{ancestor}(X, X).$
- (3) $\text{parent}(\text{amy}, \text{bob}).$

Given the goal $\text{ancestor}(X, \text{bob})$, Prolog's search strategy is left to right and depth first on the tree of subgoals shown in Figure 4.2. Edges in that figure are labeled by the number of the clause above used by Prolog for resolution, and instantiations of variables are written in curly brackets.

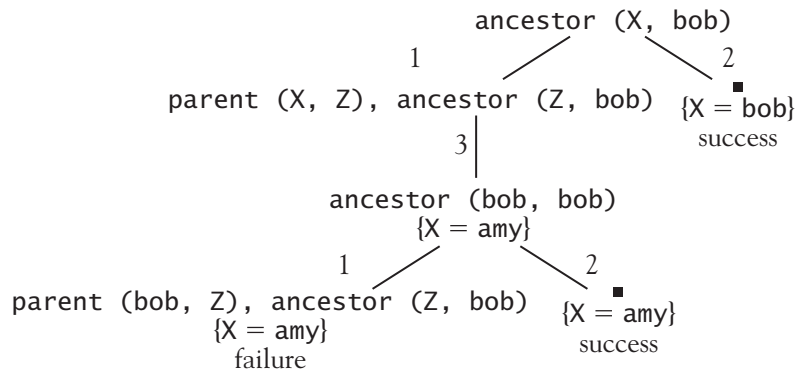


Figure 4.2 A Prolog search tree showing subgoals, clauses used for resolution, and variable instantiations

Leaf nodes in this tree occur either when no match is found for the leftmost clause or when all clauses have been eliminated, thus indicating success. Whenever failure occurs, or the user indicates a continued search with a semicolon, Prolog **backtracks** up the tree to find further paths to a leaf, releasing instantiations of variables as it does so. Thus, in the tree shown, after the solution $X = \text{amy}$ is found, if backtracking is initiated, this instantiation of X will be released, and the new path to a leaf with $X = \text{bob}$ will be found.

This depth-first strategy is extremely efficient, because it can be implemented in a stack-based or recursive fashion. However, it also means that solutions may not be found if the search tree has branches of infinite depth. For example, suppose we had written the clauses in a slightly different order:

- (1) `ancestor(X, Y) :- ancestor(Z, Y), parent(X, Z).`
- (2) `ancestor(X, X).`
- (3) `parent(amy, bob).`

This causes Prolog to go into an infinite loop, attempting to satisfy `ancestor(Z, Y)`, continually reusing the first clause. Of course, this is a result of the left-recursive way the first clause was written and the fact that no other clauses precede it. In true logic programming, the order of the clauses should not matter. Indeed, a logic programming system that adopts breadth-first search instead of depth-first search will always find solutions if there are any. Unfortunately, breadth-first search is far more expensive than depth-first search, so few logic programming systems use it. Prolog always uses depth-first search.

4.4.6 Loops and Control Structures

We can use the depth-first search with backtracking of Prolog to perform loops and repetitive searches. What we must do is force backtracking even when a solution is found. We do this with the built-in predicate `fail`. As an example, we can get Prolog to print all solutions to a goal such as `append`, with no need to give semicolons to the system as follows. Define the predicate:

```
printpieces(L) :-append(X, Y, L),
                write(X),
                write(Y),
                nl,
                fail.
```

Now we get the following behavior:

```
?- printpieces([1, 2]).
[] [1, 2]
[1] [2]
[1, 2] []
no
```

Backtracking on failure forces Prolog to write all solutions at once.

We can also use this technique to get repetitive computations. For example, the following clauses generate all integers ≥ 0 as solutions to the goal `num(X)`:

- (1) `num(0).`
- (2) `num(X) :- num(Y), X is Y + 1.`

The search tree is displayed in Figure 4.3. In this tree, it has an infinite branch to the right. (The different uses of `Y` from clause (2) in the tree are indicated by adding quotes to the variable name.)

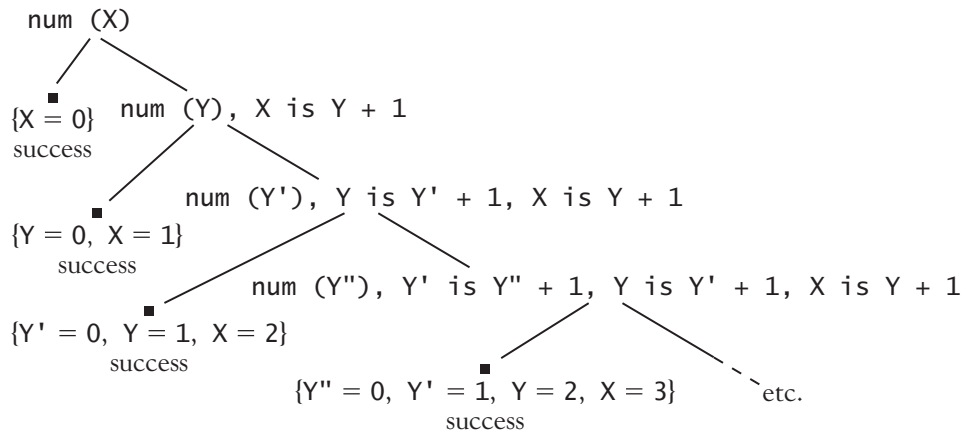


Figure 4.3 An infinite Prolog search tree showing repetitive computations

Now we could try to generate the integers from 1 to 10, say, by writing:

```
writenum(l, J) :- num(X),
                  I =< X,
                  X =< J,
                  write(X),
                  nl,
                  fail.
```

and giving the goal `writenum(1, 10)`. Unfortunately, this will go into an infinite loop after $X = 10$, generating ever-larger integers X , even though $X \leq 10$ will never succeed.

What is needed is some way of stopping the search from continuing through the whole tree. Prolog has an operator to do this: the **cut**, usually written as an exclamation point. The cut freezes a choice when it is encountered. If a cut is reached on backtracking, the search of the subtrees of the parent node of the node containing the cut stops, and the search continues with the grandparent node. In effect, the cut prunes the search tree of all other siblings to the right of the node containing the cut.

EXAMPLE 12

Consider the tree of Figure 4.2. If we rewrite the clauses using the cut as follows:

- (1) `ancestor(X, Y) :- parent(X, Z), !, ancestor(Z, Y).`
- (2) `ancestor(X, X).`
- (3) `parent(amy, bob).`

then only the solution $x = \text{amy}$ will be found, since the branch containing $x = \text{bob}$ will be pruned from the search, as shown in Figure 4.4.

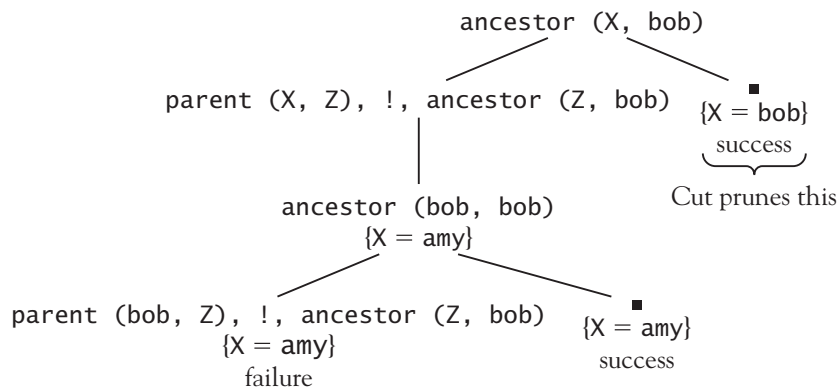


Figure 4.4 Consequences of the cut for the search tree of Figure 4.2

On the other hand, if we place the cut as follows,

- (1) `ancestor(X, Y) :- !, parent(X, Z), ancestor(Z, Y).`
- (2) `ancestor(X, X).`
- (3) `parent(amy, bob).`

then no solutions at all will be found, as shown in Figure 4.5.

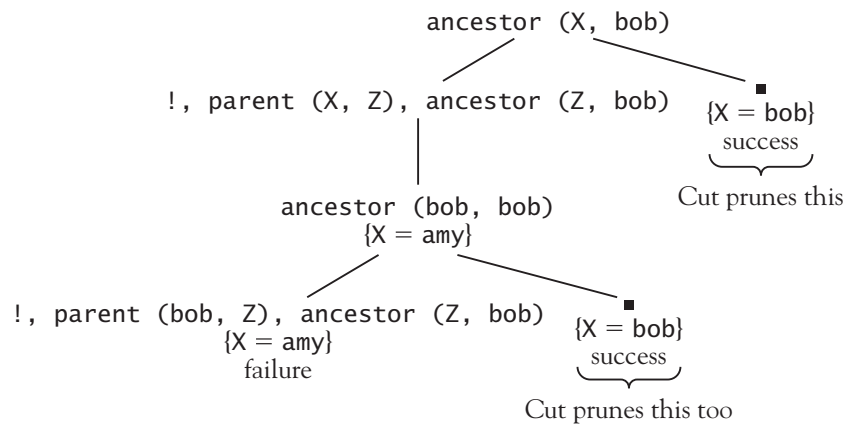


Figure 4.5 A further use of the cut prunes all solutions from Figure 4.2

Finally, if we place the cut as follows,

- (1) `ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).`
- (2) `ancestor(X, X) :- !.`
- (3) `parent(amy, bob).`

both solutions will still be found, since the right subtree of `ancestor(X, bob)` is not pruned (in fact nothing at all is pruned in this example).

The cut can be used as an efficiency mechanism to reduce the number of branches in the search tree that need to be followed. The cut also allows us to solve the problem of the infinite loop in the program to print the numbers between *I* and *J*. The following is one solution:

```
num(0).
num(X) :- num(Y), X is Y + 1.
writenum(I, J) :- num(X),
                  I =< X,
                  X =< J,
                  write(X), nl,
                  X = J, !,
                  fail.
```

In this code, `X = J` will succeed when the upper-bound *J* is reached, and then the cut will cause backtracking to fail, halting the search for new values of *X*.

The cut can also be used to imitate `if-else` constructs in imperative and functional languages. To write a clause such as:

$$D = \text{if } A \text{ then } B \text{ else } C$$

we write the following Prolog:

```
D :- A, !, B.
D :- C.
```

Note that we could have achieved almost the same result without the cut,

```
D :- A, B.
D :- not(A), C.
```

but this is subtly different, since *A* is executed twice. Of course, if *A* has no side effects, then the two forms are equivalent. Nevertheless, the cut does improve the efficiency.

Finally, we give in Figure 4.6 a longer program that uses most of the features discussed so far. It is a program to compute primes using the sieve of Eratosthenes and is adapted from an example in Clocksin and Mellish [1994].

```
primes(Limit, Ps) :- integers(2, Limit, Is),
                      sieve(Is, Ps).
integers(Low, High, [Low|Rest]) :-
    Low <= High, !, M is Low+1,
    integers(M, High, Rest).
integers(Low, High, []).
sieve([], []).
sieve([I|Is], [I|Ps]) :- remove(I, Is, New),
                        sieve(New, Ps).
remove(P, [], []).
remove(P, [I|Is], [I|Nis]) :-
    not(0 is I mod P), !, remove(P, Is, Nis).
remove(P, [I|Is], Nis) :-
    0 is I mod P, !, remove(P, Is, Nis).
```

Figure 4.6 The Sieve of Eratosthenes in Prolog (adapted from Clocksin and Mellish [1994])

4.5 Problems with Logic Programming

The original goal of logic programming was to make programming into a specification activity—that is, to allow the programmer to specify only the properties of a solution and to let the language implementation provide the actual method for computing the solution from its properties. This is what is meant by **declarative programming**, which takes the perspective that a program describes *what* a solution to a given problem is, not *how* that problem is solved. Logic programming languages, and Prolog in particular, have only partially met this goal. Instead, the nature of the algorithms used by logic programming systems, such as resolution, unification, and depth-first search, have introduced many pitfalls into the specifics of writing programs. As a programmer, you must be aware of them to write efficient, or even correct, programs. As we have already seen, it is also necessary to view a Prolog program as a set of procedure calls, which must be placed in a certain sequence in order to avoid infinite loops or inefficient execution. Moreover, the Prolog programmer must also occasionally take an even lower-level perspective of a program, as when exploiting the underlying backtracking mechanism to implement a cut/fail loop.

In this section, we discuss some of the more subtle problems relating to the adequacy of Prolog for representing logic. These include the occur-check problem in unification, problems with negation (the `not` operator), the limitations of Horn clauses in logic, and the need for control information in a logic program.

4.5.1 The Occur-Check Problem in Unification

The unification algorithm used by Prolog is actually incorrect: When unifying a variable with a term, Prolog does not check whether the variable itself occurs in the term it is being instantiated to. This is known as the occur-check problem. The simplest example is expressed by the following clause:

```
is_own_successor :- X = successor(X).
```

This will be true if there exists an X for which X is its own successor. However, even in the absence of any other clauses for `successor`, Prolog still answers yes. In other words, any X will do! That this is incorrect becomes apparent only if we make Prolog try to print such an X , as with:

```
is_own_successor(X) :- X = successor(X).
```

Now Prolog will respond with an infinite loop:

```
?- is_own_successor(X).
X = successor(successor(successor(successor(
    successor(successor(successor(successor(
    successor(successor(successor(successor(
    successor(successor(successor(successor(
    successor(successor(successor(successor(
    successor(successor(successor(successor(
    successor(successor(successor(successor
    . . . .
```

This occurs because unification has constructed X as a circular structure indicated by the picture shown in Figure 4-7.

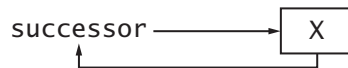


Figure 4-7: Circular structure created by unification

X is finite, but the attempt to print X is infinite. Thus, what should be logically false now becomes a programming error.

Why doesn't Prolog's unification contain a check for such occurrences? The answer is that unification without the occur-check can be relatively easily implemented in an efficient way, while efficient algorithms that include the occur-check are more complex. (For a discussion, see Lloyd [1984, p. 23].)

4.5.2 Negation as Failure

All logic programming systems have the following basic property: something that cannot be proved to be true is assumed to be false. This is called the **closed-world assumption**. We have already seen an example of this in Example 1, where we noted that, since `natural(21)` cannot be proved from the axioms for the predicate `natural`, it is assumed to be false. This property of logic programming, together with the way negation is implemented, results in further surprises in the behavior of logic programs.

How can one implement the not operator in logic programming? The straightforward answer is that the goal `not(X)` succeeds whenever the goal `X` fails. This is what is meant by **negation as failure**. As a simple example, consider the following program consisting of one clause:

```
parent(amy, bob).
```

If we now ask:

```
?- not(mother(amy, bob)).
```

the answer is **yes**, since the system does not know that amy is female, and that female parents are mothers. If we were to add these facts to our program, `not(mother(amy, bob))` would no longer be true.

The foregoing property of logic programs—that adding information to a system can reduce the number of things that can be proved—is called **nonmonotonic reasoning** and is a consequence of the closed-world assumption. Nonmonotonic reasoning has been the subject of considerable study in logic programming and artificial intelligence. (See the Notes and References.)

A related problem is that failure causes instantiations of variables to be released by backtracking, so that after failure, a variable may no longer have an appropriate value. Thus, for example, `not(not(X))` does not have the same result as `X` itself (we assume the fact `human(bob)` in this example):

```
?- human(X).
X = bob

?- not(not(human(X))).
X = _23
```

The goal `not(not(human(X)))` succeeds because `not(human(X))` fails, but when `not(human(X))` fails, the instantiation of `X` to `bob` is released, causing `X` to be printed as an uninstantiated variable.

A similar situation is shown by the following:

```
?- X = 0, not(X = 1).
X = 0

?- not (X = 1), X = 0.
no
```

The second pair of goals fails because `X` is instantiated to `1` to make `X = 1` succeed, and then `not (X = 1)` fails. The goal `X = 0` is never reached.

4.5.3 Horn Clauses Do Not Express All of Logic

Not every logical statement can be turned into Horn clauses. In particular, statements involving quantifiers may not be expressible in Horn clause form. A simple example is the following:

$p(a)$ and (there exists x , $\text{not}(p(x))$).

We can certainly express the truth of $p(a)$ as a Horn clause, but the second statement cannot be written in Horn clause form. If we try to do so in Prolog, we might try something like:

```
p(a).
not(p(b)).
```

but the second statement will result in an error (trying to redefine the not operator). Perhaps the best approximation would be simply the statement $p(a)$. Then, the closed-world assumption will force $\text{not}(p(X))$ to be true for all X not equal to a , but this is really the logical equivalent of:

$$p(a) \text{ and } (\text{for all } x, \text{not}(x = a) \rightarrow \text{not}(p(a))).$$

which is not the same as the original statement.

4.5.4 Control Information in Logic Programming

We have already talked about the cut as a useful, even essential, explicit control mechanism in Prolog. Because of its depth-first search strategy, and its linear processing of goals and statements, however, Prolog programs also contain implicit information on control that can easily cause programs to fail. One example is the ancestor program used earlier:

```
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).
ancestor(X, X).
parent(amy, bob).
```

If we accidentally wrote the right-hand side of the first clause backward, as:

```
ancestor(X, Y) :- ancestor(Z, Y), parent(X, Z).
```

we would immediately get into an infinite loop: Prolog will try to instantiate the variable Z to make ancestor true, causing an infinite descent in the search tree using the first clause. On the other hand, if we changed the order of the clauses to:

```
ancestor(X, X).
ancestor(X, Y) :- ancestor(Z, Y), parent(X, Z).
parent(amy, bob).
```

the search would now find both solutions $X = \text{amy}$ and $X = \text{bob}$ to the query $\text{ancestor}(\text{amy}, X)$, but would still go into an infinite loop searching for further (nonexistent) solutions. A similar situation exists if we write the clauses for the program to generate natural numbers in reverse order:

```
num(X) :- num(Y), X is Y + 1.
num(0).
```

Now the goal $\text{num}(X)$ will go into an infinite loop. Nothing at all will be generated.

A more complicated logical question is the representation of algorithmic logic using Horn clauses. We mentioned in Section 4.3 that a specification for a sorting procedure could be given as follows, for two lists S and T :

```
sort(S, T) :- permutation(S, T), sorted(T).
```

We can now give specifications for the meaning of permutation and sorted in Prolog syntax, in Figure 4.8.

```
sorted([]).
sorted([X]).
sorted([X, Y|Z]) :- X <= Y, sorted([Y|Z]).

permutation([], []).
permutation(X, [Y|Z]) :- append(U, [Y|V], X),
                        append(U, V, W),
                        permutation(W, Z).
```

Figure 4.8 The definitions of the `permutation` and `sorted` properties in Prolog

This represents a mathematical definition for what it means for a list of numbers to be sorted in increasing order. But as a program, it represents almost the slowest sort in the world. Permutations of the unsorted list are generated until one of them happens to be sorted!

In the best of all possible worlds, one would want a logic programming system to accept the mathematical definition of a property and find an efficient algorithm to compute it. Of course, in Prolog or any other logic programming system, we not only provide specifications in our programs, but we must also provide algorithmic control information. Thus, in the `gcd` program of Example 10, we specified explicitly the steps used by Euclid's algorithm instead of providing a mathematical definition of the greatest common divisor. Similarly, to get a reasonably efficient sorting program in Prolog, we must specify the actual steps an algorithm must take to produce the result. For example, a `quicksort` program in Prolog is given in Figure 4.9.

```
qsort([], []).
qsort([H|T], S) :- partition(H, T, L, R),
                  qsort(L, L1),
                  qsort(R, R1),
                  append(L1, [H|R1], S).
partition(P, [A|X], [A|Y], Z) :- A < P,
                                partition(P, X, Y, Z).
partition(P, [A|X], Y, [A|Z]) :- A >= P,
                                partition(P, X, Y, Z).
partition(P, [], [], []).
```

Figure 4.9 A `quicksort` program in Prolog (adapted from Clocksin and Mellish [1994])

In this program, the standard method of partitioning is shown by the clauses for the predicate `partition`, and the two recursive calls to `qsort` in the second clause are readily evident. Thus, in specifying sequential algorithms such as sorting, Prolog is not so different from imperative or functional programming as one might think.

There is, however, one useful aspect to being able to execute a specification for a property, even if it is not acceptable as an algorithm for computing that property: It lets us test whether the specification is correct. Prolog has in fact been used successfully as a basis for running such specifications during the design of a program. (See the Notes and References.)

4.6 Curry: A Functional Logic Language

In this chapter and the previous one, we have examined two styles or paradigms of programming, the functional and the logical, and some languages that support them. By using these paradigms instead of the more conventional imperative paradigm, a programmer can more or less write an executable specification of a program—that is, describe what a program does to solve a problem and leave much of the implementation, or how the solution will be carried out, to the underlying interpreter of the language. Thus, such languages support a **declarative style** of programming, as distinct from the more conventional procedural style associated with imperative languages. Each paradigm, whether functional or logical, nevertheless brings something different to the declarative enterprise. In a functional language, a program is a set of function definitions, which specify rules for operating on data to transform it into other data. In a logic language, a program is a set of rules and facts from which a proof is constructed for a solution to a problem. We have also seen that each of these versions of the declarative paradigm has some specific disadvantages. Logic programming often forces the programmer to abandon the declarative perspective when solving some problems, such as numeric calculations, that require a functional perspective, in which a set of inputs is mapped to a single output. Functional programming imposes the opposite restriction on the programmer. A function, which deterministically maps a given set of arguments to a single value, is not well suited for problems that involve determining a set of values that a variable can have that satisfy certain constraints and that typically are computed in a nondeterministic fashion.

In this section, we give a short overview of the language Curry, which brings together the advantages of functional and logic programming in a single language. Our discussion is an adaptation, with examples, of the work reported by Antoy and Hanus [2010].

4.6.1 Functional Programming in Curry

Curry is an extension of the programming language Haskell discussed in Chapter 3. Both languages are named after the mathematician Haskell Curry. Curry retains the syntax and semantics of Haskell for functional programming, and adds new syntax and semantics for logic programming. As in Haskell, a functional program in Curry consists of a set of function definitions that describe transformations of data values into other data values. Pattern matching, case analysis, recursion, and strong, polymorphic typing are critical aspects of the language. As you saw in Chapter 3, function definitions are sets of equations, whose interpretation involves replacing the pattern on the left side of the = symbol with the pattern on the right side, after replacing the variables in the patterns with the actual parameters. Curry also uses lazy evaluation, whereby the values of actual parameters are computed only when needed.

4.6.2 Adding Nondeterminism, Conditions, and Backtracking

A pure functional language supports only deterministic computation. This means that the application of a function to a given set of arguments always produces the same value. However, some problems, such as flipping a coin, are underspecified, in that their solutions come from a set of values that need only satisfy some constraints. In the case of a coin toss, a definite value is returned, but it will be either heads or tails. Curry supports such nondeterminism by allowing a set of equations for a function to be tried in no particular order. For example, the choice operator `?` is defined with two equations:

```
x ? y = x
x ? y = y
```

When `?` is applied to some operands, Curry, unlike Haskell, does not automatically try the first equation that appears in the definition, which would always return the first operand. Moreover, if one equation fails during execution, another equation in the set is then tried, although that won't happen in this particular example. Thus, a nondeterministic function for flipping a coin can be defined using `?`:

```
flipCoin = 0 ? 1
```

In Section 4.5.4, we showed some Prolog rules for sorting a list that generated permutations of a list until one of them satisfied the constraint of being sorted. This sorting strategy was clear and simple but suffered the woeful inefficiency of generating permutations of the list until one of them satisfied the `sorted` predicate. A similar, but more efficient, strategy can be used to define functions to sort a list in Curry.

Working top-down, we define two functions, `sorted` and `permutation`, such that the application:

```
sorted (permutation xs)
```

returns a sorted list of the elements in `xs`. Obviously, if the list returned by a call of `permutation` is not sorted, the computation must somehow backtrack and try another call of the function, until it produces a sorted list. The strategy used here exploits Curry's support for nondeterminism and backtracking to produce a simple implementation.

The function `sorted` expects a list as an argument and returns a sorted list of the same elements. The first two equations in its definition specify the results for an empty list and a list of one element:

```
sorted [] = []
sorted [x] = [x]
```

The third equation handles the case of a list of two or more elements. In this case, the equation includes a **condition** (specified by the code to the right of the `|` symbol) that permits evaluation of its right side only if the first element in the list is less than or equal to the second element:

```
sorted (x:y:ys) | x <= y
                = x : sorted(y:ys)
```

As you can see, this function will succeed and return a sorted list just when it receives a permutation of the list that happens to be sorted. However, it fails as soon as the first two elements of the list argument are out of order.

The function `permutation` inserts the first element of a nonempty list argument into a permutation of the rest of that list:

```
permutation []      = []
permutation (x:xs) = insert x (permutation xs)
```

The function `insert` places an element at an arbitrary position in a list. The function is defined nondeterministically for nonempty lists, in the following pair of equations:

```
insert x ys      = x : ys
insert x (y:ys) = y : insert x ys
```

This code is simple and elegant enough, but two questions remain to be answered. First, how does Curry back up and call for another permutation of the list if the `sorted` function fails? Second, why is this version of the sorting strategy more efficient than the Prolog version in Section 4.5.4?

To answer the first question, let's assume that the function `sorted`, at some point, fails. This means that the first two elements of its argument, a list permutation, have been found to be out of order. In Curry, control then returns to the definition of `sorted`, to try an equation whose formal parameter pattern matches that of the actual parameter. This pattern-matching step requires another call of the function `permutation`, however, to determine whether the actual parameter is a list of zero, one, or two or more elements. Also, because `permutation` is at bottom nondeterministic, it stands a good chance of returning a different ordering of elements in the list than on its previous call. Even if two consecutive identical lists are examined, this process of generating and testing will continue until a sorted list is returned.

The answer to the second question, which concerns the inefficiency of generating permutations in this strategy, lies in understanding the advantage of lazy evaluation. Recall that Curry, like Haskell, evaluates only enough information in a function's parameters to be able to perform the required computations at each step. As you saw in the discussion of list processing in Section 3.4, this means that functions that supply lists to other functions need only provide the front-end portions that are needed for immediate computations. This allows for the representation of infinite lists and a generate-and-filter model of processing. In the present case, the `sorted` function need only receive from the call of the `permutation` function at most the first two elements of a list, in order to be in a position to fail or continue demanding more. The fact that the Curry-based strategy can fail early gives it an efficiency advantage over the Prolog-based strategy, which must complete each permutation of a list before it can be tested.

4.6.3 Adding Logical Variables and Unification

Two other elements of logic programming, logical variables and unification, give Curry the ability to solve equations with unknown or partial information. This process involves viewing some variables as free—not in the sense of the free variables considered in Chapter 3, which are always bound at the time they are referenced within a function—but free in the sense that they can be instantiated in a manner that satisfies a set of equations that includes them. Curry uses the symbol `==` instead of the symbol `=` to specify an equation that should be solved in this manner, rather than an equation that defines a function application. For example, the equation:

```
zs ++ [2] == [1, 2]
```

uses the list concatenation operator `++` to solve for the variable `zs`. It does so by instantiating the variable `zs` with the only value, `[1]`, that makes this equation true. Lazy evaluation is put to work again with free variables, which are instantiated only when their values are needed to apply a function. Thus, to evaluate the previous equation, Curry chooses a value for `zs` that allows one of the equations for the definition of `++`:

```
[ ]      ++ ys = ys
(x:xs) ++ ys = x : (xs ++ ys)
```

to be applied. This value will be either `[ ]`, for the first equation, or a value of the form `(x:xs)`, for the second one. The first rule does not succeed, because `[ ] ++ [2]` does not equal `[1, 2]`. However, the second equation does succeed, with the bindings of `1` for `x`, `[ ]` for `xs`, and `[2]` for `ys`.

As in Prolog, logical variables can remain uninstantiated if not enough information is available. They can also be used to obtain multiple solutions to the same equation, as in:

```
first ++ second == [1, 2]
```

Finally, logical variables can be used in the definition of new functions. For example, the equation:

```
zs ++ [e] == [1, 2, 3]
```

is satisfied for `zs = [1, 2]` and `e = 3`. Using this information, we can define a function `last` that extracts the last element in a nonempty list, as follows:

```
last xs | zs ++ [e] == xs
      = e
      where zs, e free
```

This equation includes a condition with an equation that contains two free variables, as well as the argument to the function. The outer equation essentially generalizes the inner equation for any nonempty list.

Exercises

4.1 A standard method for analyzing logical statements is the truth table: Assigning truth values to elementary statements allows us to determine the truth value of a compound statement, and statements with the same truth tables are logically equivalent. Thus the statement “*p* and not *p*” is equivalent to “false” by the following truth table:

<i>p</i>	not <i>p</i>	<i>p</i> and not <i>p</i>	false
false	true	false	false
true	false	false	false

Use truth tables to show that $p \rightarrow q$ is equivalent to $(\text{not } p) \text{ or } q$ (remember that $p \rightarrow q$ is false only if p is true and q is false).

4.2 A **tautology** is a statement that is always true, no matter what the truth values of its components. Use a truth table to show that $\text{false} \rightarrow p$ is a tautology for any statement p .

- 4.3** A **refutation system** is a logical system that proves a statement by assuming it is false and deriving a contradiction. Show that Horn clause logic with resolution is a refutation system. (*Hint:* The empty clause is assumed to be false, so a goal $\leftarrow a$ is equivalent to $a \rightarrow \text{false}$. Show that this is equivalent to $\text{not}(a)$.)
- 4.4** Write the following statements in the first-order predicate calculus:
 If it is raining or snowing, then there is precipitation.
 If it is freezing and there is precipitation, then it is snowing.
 If it is not freezing and there is precipitation, then it is raining.
 It is snowing.
- 4.5** Write the statements in Exercise 4.4 as Prolog clauses, in the order given. What answer does Prolog give when given the query “Is it freezing?” The query “Is it raining?” Why? Can you rearrange the clauses so that Prolog can give better answers?
- 4.6** Write the following mathematical definition of the greatest common divisor of two numbers in first-order predicate calculus: the gcd of u and v is that number x such that x divides both u and v , and, given any other number y such that y divides u and v , then y divides x .
- 4.7** Translate the definition of the gcd in Exercise 4.6 into Prolog. Compare its efficiency to Euclid’s gcd algorithm as given in Figure 4.1.
- 4.8** Write the following statement as Prolog clauses: Mammals have four legs and no arms, or two arms and two legs.
- 4.9** Add the statement that a horse is a mammal and that a horse has no arms to the clauses of Exercise 4.8. Can Prolog derive that a horse has four legs? Explain.
- 4.10** Write Prolog clauses to express the following relationships, given the parent relationship: grandparent, sibling, cousin.
- 4.11** Write a Prolog program to find the last item in a list.
- 4.12** Write a Prolog program to find the maximum and minimum of a list of numbers.
- 4.13** Write a Prolog program that reads numbers from the standard input until a 0 is entered, creates a list of the numbers entered (not including the 0), and then prints the list in the order entered, one number per line. (`read(X)` can be used to read integer X from the input if the integer is entered on its own line and is terminated by a period.)
- 4.14** Write a Prolog program that will sort a list of integers according to the mergesort algorithm.
- 4.15** Compare the Prolog program for quicksort in Figure 4.9 with the Haskell version in Section 3.5. What are the differences? What are the similarities?
- 4.16** Prolog shares some features with functional languages like Scheme, ML, and Haskell, studied in Chapter 3. Describe two major similarities. Describe two major differences.
- 4.17** In Prolog it is possible to think of certain clauses as representing tail recursion, in which the final term of a clause is a recursive reference, and a cut is written just before the final term. For example, the gcd clause
- $$\text{gcd}(U, V, W) \text{ :- not}(V=0), R \text{ is } U \bmod V, !, \text{gcd}(V, R, W).$$
- can be viewed as being tail recursive. Explain why this is so.
- 4.18** Write a Prolog program to print all Pythagorean triples (x, y, z) such that $1 \leq x, y \leq z \leq 100$. ((x, y, z) is a Pythagorean triple if $x^2 + y^2 = z^2$.)

- 4.19** Write Prolog clauses for a member predicate: `member(X,L)` succeeds if `X` is a member of the list `L` (thus `member(2,[2,3])` succeeds but `member(1,[2,3])` fails). What happens if you use your clauses to answer the query `member(X,[2,3])`? The query `member(2,L)`? Draw search trees of subgoals as in Section 4.4.5 to explain your answers.

- 4.20** Rewrite the factorial Prolog program

```
fact(0, 1).
fact(N, R) :-
    N > 0, N1 is N - 1, fact(N1, R1), R is N * R1.
```

to make it tail recursive (see Exercise 4.17).

- 4.21** Draw search trees of subgoals and explain Prolog's responses based on the trees for the following goals (see Figure 4.1):

- (a) `gcd(15, 10, X)`.
 (b) `append(X, Y, [1, 2])`.

- 4.22** Rewrite the Prolog clauses for Euclid's algorithm (`gcd`) in Figure 4.1 to use the cut instead of the test `not(V = 0)`. How much does this improve the efficiency of the program? Redraw the search tree of Exercise 4.21(a) to show how the cut prunes the tree.

- 4.23** Explain using a search tree why the cut in the following program has no effect on the solutions found to the query `ancestor(X,bob)`. Does the cut improve the efficiency at all? Why or why not?

```
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).
ancestor(X, X) :- !.
parent(amy, bob).
```

- 4.24** If we used cuts to improve the efficiency of the Prolog `append` program, we would write:

```
append([], Y, Y) :- !.
append([A|B], Y, [A|W]) :- append(B, Y, W).
```

Now given the goal `append(X, Y, [1, 2])` Prolog only responds with the solution `X = [], Y = [1, 2]`. Explain using a search tree.

- 4.25** Rewrite the sieve of Eratosthenes Prolog program of Figure 4.6 to remove all the cuts. Compare the efficiency of the resulting program to the original.

- 4.26** Explain the difference in Prolog between the following two definitions of the sibling relationship:

```
sibling1(X, Y) :- not(X = Y), parent(Z, X), parent(Z, Y).
sibling2(X, Y) :- parent(Z, X), parent(Z, Y), not(X = Y).
```

Which definition is better? Why?

- 4.27** Given the following Prolog clauses:

```
ancestor(X, Y) :- ancestor(Z, Y), parent(X, Z).
ancestor(X, X).
parent(amy, bob).
```

explain, using a search tree of subgoals, why Prolog fails to answer when given the goal `ancestor(X,bob)`.

- 4.28 Given the following Prolog clauses:

```
ancestor(X, X).
ancestor(X, Y) :- ancestor(Z, Y), parent(X, Z).
parent(amy, bob).
```

explain Prolog's response to the query `ancestor(amy,X)` using a search tree of subgoals.

- 4.29 What is wrong with the following Prolog specification for a sort procedure?

```
sort(S, T) :- sorted(T), permutation(S, T).
```

Why?

- 4.30 Given only the following Prolog clause:

```
human(bob).
```

Prolog will respond as follows:

```
?- human(X).
X = bob

?- not(human(X)).
no
```

Why did Prolog respond to the last goal as it did? Is Prolog saying that there are no X that are not human, that is, that all X are human? Why?

- 4.31 The following is a possible implementation of a for loop construct in Prolog, similar to the statement `for(I = L; I <= H; I++)` in C:

```
for(I, I, I) :- !.
for(I, I, H) .
for(I, L, H) :- L1 is L + 1, for(I, L1, H).
```

Using this definition, we can repeat operations over I , such as printing all integers between L and H as follows:

```
printint(L, H) :- for(I, L, H), write(I), nl, fail.
```

Explain how backtracking causes the `for` predicate to behave like a loop.

- 4.32 In the brief explanation of the difference between free and bound variables in Section 4.1, the details about potential reuse of names were ignored. For example, in the statement:

$$a(x) \rightarrow \text{there exists } x, b(x)$$

the x in b is bound, while the x in a is free. Thus, the **scope of a binding** must be properly defined to refer to a subset of the uses of the bound variable. Develop scope rules for bindings from which it can be determined which uses of variables are free and which are bound.

- 4.33 Compare the definition of free and bound variables in logical statements with the definition of free and bound variables in the lambda calculus (Chapter 3). How are they similar? How are they different?

4.34 The occur-check was left out of Prolog because simple algorithms are inefficient. Describe using the append clauses of Figure 4.1 why the occur-check can be inefficient.

4.35 We have seen that Prolog can have difficulties with infinite data sets, such as that produced by the clauses

```
int(0).
int(X) :- int(Y), X is Y + 1 .
```

and with self-referential terms such as:

```
X = [1|X]
```

which causes an occur-check problem. In Haskell (Chapter 3), infinite data constructions, such as the foregoing, are possible using delayed evaluation. Does delayed evaluation make sense in Prolog? Can you think of any other ways infinite data structures might be dealt with?

4.36 Kowalski [1988] makes the following statement about his pseudoequation $A(\text{lgorithm}) = L(\text{ogic}) + C(\text{ontrol})$: “With Prolog we have a fixed C and can improve A only by improving L . Logic programming is concerned with the possibility of changing both L and C .” Explain why he says this. Explain why it is not completely true that Prolog cannot change C .

4.37 In Chapter 3, we described a unification method used by ML and Haskell to perform type inference. Compare this unification with the unification of Prolog. Are there any differences?

4.38 Unification can be described as a substitution process that substitutes more specific values for variables so that equality of two expressions is achieved. For example, in Prolog the lists $[1|Y] = [X]$ can be unified by substituting 1 for X and $[]$ for Y . It is possible for different substitutions to cause equality. For instance, if $[1|Y]$ is unified with X , one could set $X = [1]$ and $Y = []$, or one could set $X = [1|Y]$ and leave Y uninstantiated. This latter substitution is **more general** than the first because the additional substitution $Y = []$ transforms the first into the second. A substitution is a **most general unifier** of two expressions if it is more general than every other substitution that unifies the two expressions. Why does the unification algorithm of Prolog produce a most general unifier? Would it be useful for a logic programming language to have a unification algorithm that does not produce a most general unifier? Why?

Notes and References

Logic programming arose out of work on proof techniques and automated deduction. The resolution principle was developed by Robinson [1965], and Horn clauses were invented by Horn [1951]. Kowalski [1979b] gives a general introduction to logic and the algorithms used in logic programming, such as unification and resolution. Lloyd [1984] gives a mathematical treatment of general unification techniques and negation as failure.

The early history of the development of Prolog is discussed in a pair of companion articles by Kowalski [1988] and Cohen [1988]. Both mention the importance of the first interpreters developed by Alain Colmerauer and Phillipe Roussel in Marseilles and the later systems written by David Warren, Fernando Pereira, and Luis Pereira in Edinburgh. For a further perspective, see Colmerauer and Roussel [1996].

For a perspective and overview of the Japanese Fifth Generation Project (1981–1992), see Shapiro and Warren [1993].

Kowalski [1979a] introduced the famous formulation of Prolog’s basic strategy as “Algorithm = Logic + Control” and gives a detailed account of how computation can be viewed as theorem proving, or “controlled deduction.” (Niklaus Wirth’s formulation of imperative programming “Algorithms + Data Structures = Programs” is the title of Wirth [1976] and is discussed in Chapters 1 and 8.)

Clocksin and Mellish [1994] is a basic reference for Prolog programming. Some of the examples in this chapter are based on examples in that text. Other references for programming techniques in Prolog are Sterling and Shapiro [1986], Malpas [1987], and Bratko [2000]. A Prolog standard was adopted by the ISO and ANSI in 1995 (ISO 13211-1 [1995]).

Davis [1982] describes how Prolog can be used as a runnable specification in the design of a software system. Warren [1980] shows how Prolog can be used in language translation and compilers. Clocksin and Mellish [1994] provide a chapter on the use of Prolog to parse grammars and a chapter on the relation of logic programming to logic, in particular the translation of statements in predicate calculus into Horn clauses. For an approach to infinite data structures in Prolog (Exercise 4.35), see Colmerauer [1982].

Nonmonotonic reasoning, mentioned in Section 4.5.2, has been the subject of considerable ongoing research; see for example Minker [2000] or Antoniou and Williams [1997].